

 Contents[Overview](#)[Architecture](#)[Getting Started](#)[API Reference](#)[Web Dashboard](#)[Users, Groups & Quotas](#)[Service API Keys](#)[Backend Management](#)[Scheduling & Fair Share](#)[Retry & Failover](#)[Circuit Breaker](#)[Rate Limiting](#)[Tool Calling](#)[Document OCR](#)[Web Search](#)[MCP Servers](#)[Agent Skills](#)[Request ID](#)[Translation Layer](#)[Telemetry & Monitoring](#)[Data Retention](#)[Chat System](#)[Voice \(TTS / STT\)](#)[Blog / CMS](#)[Email System](#)[AppConfig System](#)[Configuration Reference](#)[Status Enums Reference](#)

MindRouter Documentation

MindRouter is a production-ready LLM inference load balancer and translation layer that fronts a heterogeneous cluster of Ollama and vLLM inference backends. It provides a unified OpenAI-compatible API surface with native Ollama compatibility, fair-share scheduling, per-user quotas, full audit logging, and real-time GPU telemetry.

Developed by Luke Sheneman, Research Computing and Data Services (RCDS), Institute for Interdisciplinary Data Sciences (IIDS), University of Idaho.

Overview

MindRouter sits between API consumers and GPU inference servers, providing:

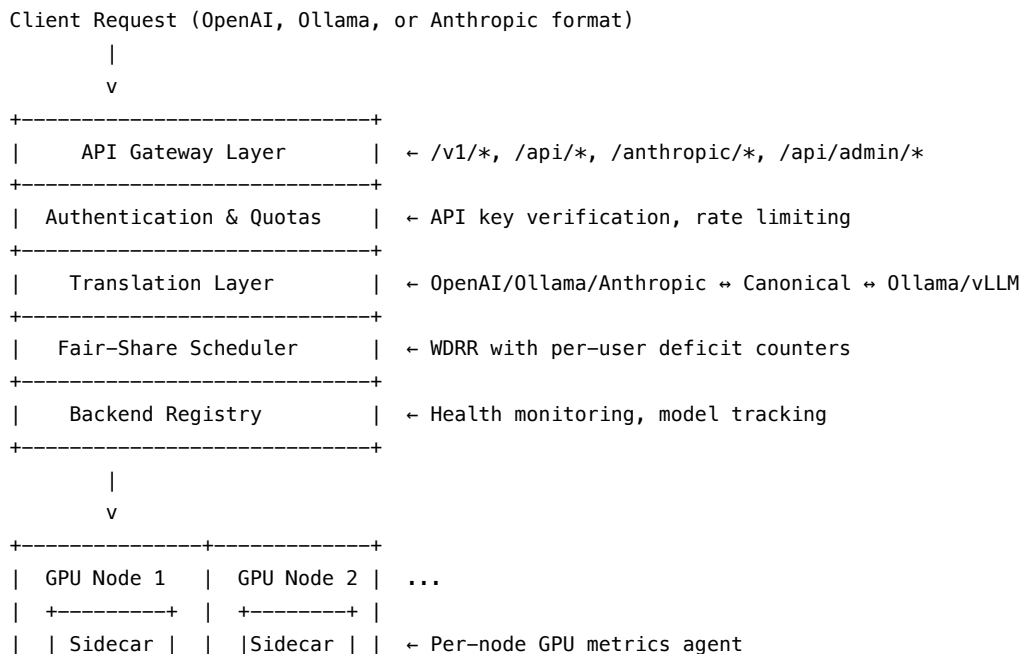
- **Unified API Gateway** — OpenAI-compatible `/v1/*`, Ollama-compatible `/api/*`, and Anthropic-compatible `/anthropic/v1/*` endpoints, all backed by the same pool of inference servers.
- **Cross-Engine Routing** — A request arriving in OpenAI, Ollama, or Anthropic format can be served by any backend. The translation layer handles all protocol conversion transparently.
- **Fair-Share Scheduling** — Weighted Deficit Round Robin (WDRR) ensures equitable GPU access across users in different groups with configurable priorities.
- **Multi-Modal Support** — Text chat, text completion, embeddings, multimodal models, structured JSON outputs, tool calling (function calling), and [voice \(TTS/STT\)](#).
- **Per-User Quotas** — Token budgets and requests-per-minute (RPM) rate limits, with defaults inherited from the user's group. RPM enforcement is shared across all workers via Redis.
- **Full Audit Logging** — Every prompt, response, and token count is recorded for compliance and review.
- **Real-Time GPU Telemetry** — Per-GPU utilization, memory, temperature, and power metrics via lightweight sidecar agents.
- **Web Dashboards** — Public status page, user self-service dashboard, admin control panel, and built-in chat interface.

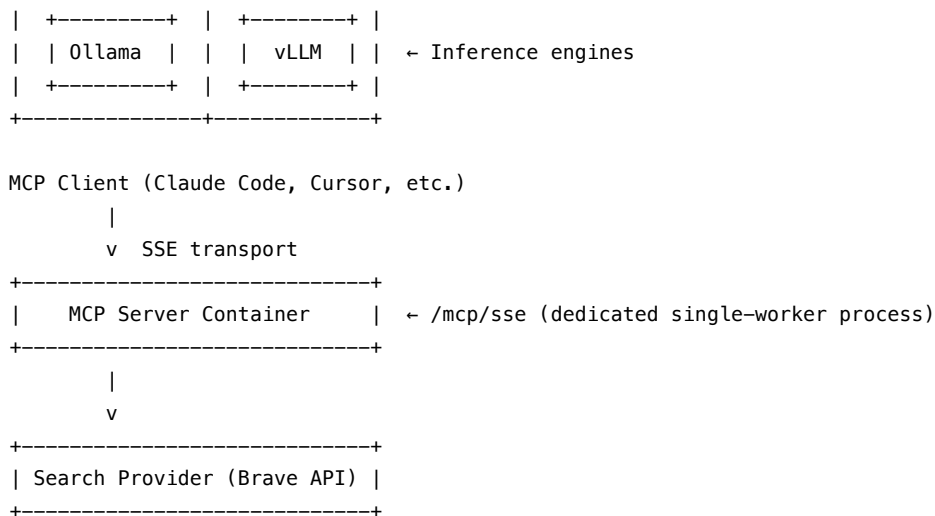
Who It's For

- **Research computing centers** managing shared GPU clusters for multiple user groups
- **Universities** providing LLM access to students, staff, and faculty with differentiated quotas
- **Organizations** needing a unified API gateway across mixed Ollama/vLLM infrastructure

Architecture

MindRouter follows a layered architecture:





Key concepts:

- A **Node** is a physical GPU server running a sidecar agent.
- A **Backend** is an inference endpoint (Ollama or vLLM instance) running on a node. Multiple backends can share a node, each assigned specific GPUs via `gpu_indices`.
- The **MCP Server** runs as a separate single-worker container, exposing MindRouter tools (e.g., web search) to MCP-compatible agents over SSE transport. The main app proxies `/mcp/*` requests to this container.

Getting Started

Prerequisites

- Docker and Docker Compose
- Python 3.11+ (for local development)

Quickstart with Docker Compose

```

# 1. Clone and configure
git clone <repository-url>
cd mindrouter
cp .env.example .env
nano .env # Set DATABASE_URL, SECRET_KEY, etc.

# 2. Start all services
docker compose up --build

# 3. Run database migrations
docker compose exec app alembic upgrade head

# 4. Seed development data (creates users, quotas, API keys)
docker compose exec app python scripts/seed_dev_data.py
  
```

Default Development Credentials

After running the seed script:

Username	Password	Group	Scheduler Weight
admin	admin123	Admin	10
faculty1	faculty123	Faculty	3
staff1	staff123	Staff	2
student1	student123	Student	1

Accessing the Application

URL	Description
http://localhost:8000/	Public status page
http://localhost:8000/dashboard	User dashboard (login required)
http://localhost:8000/admin	Admin dashboard (admin group required)
http://localhost:8000/chat	Chat interface (login required)
http://localhost:8000/docs	Interactive API docs (Swagger UI)
http://localhost:8000/redoc	API reference (ReDoc)

API Reference

Interactive API Documentation

MindRouter includes built-in interactive API documentation powered by FastAPI:

- **Swagger UI** at </docs> — Interactive API explorer where you can try endpoints directly from your browser. Supports authentication via the "Authorize" button (enter your API key as a Bearer token).
- **ReDoc** at </redoc> — Clean, readable API reference with request/response schemas and examples.

Both are auto-generated from the application's route definitions and Pydantic models, so they always reflect the current API surface.

Authentication

All inference and admin endpoints require authentication. MindRouter supports two methods:

API Key (Bearer Token):

```
curl -H "Authorization: Bearer mr2_your-api-key" http://localhost:8000/v1/models
```

API Key (Header):

```
curl -H "X-API-Key: mr2_your-api-key" http://localhost:8000/v1/models
```

Session Cookie (dashboard/admin AJAX only): Browser-based dashboard calls authenticate via the `mindrouter_session` cookie set at login. This is used internally by the web UI and is not intended for programmatic access.

Error Responses

All error responses follow a consistent format:

```
{
  "detail": "Human-readable error message"
}
```

Common HTTP status codes:

Code	Meaning
400	Invalid request body or parameters
401	Missing or invalid API key
403	Insufficient permissions (e.g., non-admin accessing admin endpoint)
404	Resource not found
409	Conflict (duplicate name, URL, etc.)
429	Rate limit exceeded
500	Internal server error

Error Response Formats by API Style

Error response formats differ depending on which API style the client is using:

- **OpenAI** (`/v1/*`) — Returns a nested error object: `{"error": {"message": "...", "type": "...", "code": ...}}`
- **Ollama** (`/api/*`) — Returns a plain detail string: `{"detail": "..."}"`
- **Anthropic** (`/anthropic/v1/*`) — Returns a typed error object: `{"type": "error", "error": {"type": "...", "message": "..."} }`

Model Name Matching

Model names in requests are matched **exactly** against the model catalog. There is no prefix matching, alias resolution, or fuzzy matching. The `model` field must match a model name as listed by `/v1/models` or `/api/tags`.

OpenAI-Compatible Endpoints

These endpoints accept and return data in the OpenAI API format. Any OpenAI-compatible client or SDK can be pointed at MindRouter by changing the base URL.

Method	Path	Auth	Description
POST	<code>/v1/chat/completions</code>	API Key	Chat completions (streaming and non-streaming)
POST	<code>/v1/completions</code>	API Key	Text completions (legacy)
POST	<code>/v1/embeddings</code>	API Key	Generate embeddings
POST	<code>/v1/rerank</code>	API Key	Rerank documents against a query
POST	<code>/v1/score</code>	API Key	Score similarity between text pairs
POST	<code>/v1/tokenize</code>	API Key	Count input tokens for a request
GET	<code>/v1/models</code>	API Key	List available models

Chat Completions

```
curl -X POST http://localhost:8000/v1/chat/completions \
```

```
-H "Authorization: Bearer mr2_your-api-key" \
-H "Content-Type: application/json" \
-d '{
  "model": "llama3.2",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello!"}
  ],
  "temperature": 0.7,
  "max_tokens": 500,
  "stream": false
}'
```

Response:

```
{
  "id": "chatcmpl-abc123...",
  "object": "chat.completion",
  "created": 1700000000,
  "model": "llama3.2",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Hello! How can I help you today?"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 25,
    "completion_tokens": 10,
    "total_tokens": 35
  }
}
```

Streaming — Set `"stream": true` to receive Server-Sent Events (SSE):

```
curl -X POST http://localhost:8000/v1/chat/completions \
-H "Authorization: Bearer mr2_your-api-key" \
-H "Content-Type: application/json" \
-d '{"model": "llama3.2", "messages": [{"role": "user", "content": "Hi"}], "stream": true}'
```

Thinking/Reasoning Mode:

MindRouter supports multiple formats for controlling thinking/reasoning on models that support it (qwen3.5, qwen3, gpt-oss):

```
// gpt-oss: control reasoning depth
{
  "model": "openai/gpt-oss-120b",
  "messages": [{"role": "user", "content": "Solve this step by step"}],
  "reasoning_effort": "high",
  "max_completion_tokens": 16384
}
```

```
// Qwen-style: toggle thinking on/off
{
  "model": "qwen/qwen3.5-400b",
  "messages": [{"role": "user", "content": "Explain quantum computing"}],
  "chat_template_kwargs": {"enable_thinking": true},
  "max_completion_tokens": 16384
}
```

When thinking is enabled, the response includes `reasoning_content` alongside `content`.

Important: Thinking models can consume large numbers of output tokens for reasoning. Use `max_completion_tokens` (or `max_tokens`) to set an adequate budget — 16384 is recommended for qwen3.5-400b with thinking enabled.

Output Token Limits:

MindRouter accepts both `max_completion_tokens` (preferred, current OpenAI standard) and `max_tokens` (legacy). If both are provided, `max_completion_tokens` takes priority.

Structured Output (JSON Mode):

```
{
  "model": "llama3.2",
  "messages": [{"role": "user", "content": "List 3 colors as JSON"}],
  "response_format": {"type": "json_object"}
}
```

Structured Output (JSON Schema):

```
{
  "model": "llama3.2",
  "messages": [{"role": "user", "content": "List 3 colors"}],
  "response_format": {
    "type": "json_schema",
    "json_schema": {
      "name": "colors",
      "schema": {
        "type": "object",
        "properties": {
          "colors": {"type": "array", "items": {"type": "string"}}
        }
      }
    }
  }
}
```

Vision (Multimodal):

```
{
  "model": "llava",
  "messages": [
    {
      "role": "user",
      "content": [
        {"type": "text", "text": "What's in this image?"},

```

```

        {"type": "image_url", "image_url": {"url": "data:image/jpeg;base64,..."}}
    ]
}
]
}

```

Tool Calling (Function Calling):

```

{
  "model": "llama3.2",
  "messages": [{"role": "user", "content": "What's the weather in Seattle?"}],
  "tools": [{
    "type": "function",
    "function": {
      "name": "get_weather",
      "description": "Get the current weather",
      "parameters": {
        "type": "object",
        "properties": {"city": {"type": "string"}},
        "required": ["city"]
      }
    }
  ]
},
  "tool_choice": "auto"
}

```

When the model decides to call a tool, the response includes `tool_calls` with `finish_reason: "tool_calls"`. Submit the tool result back as a `role: "tool"` message with the matching `tool_call_id`.

Embeddings

```

curl -X POST http://localhost:8000/v1/embeddings \
-H "Authorization: Bearer mr2_your-api-key" \
-H "Content-Type: application/json" \
-d '{"model": "nomic-embed-text", "input": "Hello world"}'

```

Reranking

```

curl -X POST http://localhost:8000/v1/rerank \
-H "Authorization: Bearer mr2_your-api-key" \
-H "Content-Type: application/json" \
-d '{
  "model": "Qwen/Qwen3-Reranker-8B",
  "query": "What is machine learning?",
  "documents": [
    "Machine learning is a subset of AI.",
    "The weather is sunny today.",
    "Deep learning uses neural networks."
  ],
  "top_n": 2
}'

```

Scoring

```

curl -X POST http://localhost:8000/v1/score \
-H "Authorization: Bearer mr2_your-api-key" \
-H "Content-Type: application/json" \

```



```
-d '{
  "model": "Qwen/Qwen3-Reranker-8B",
  "text_1": "What is machine learning?",
  "text_2": ["Machine learning is a subset of AI.", "The weather is sunny today."]
}'
```

Tokenize

Count the number of input tokens a request will consume. For **vLLM** backends, this returns an **exact** count from the backend's native tokenizer, including all chat template overhead and tool definitions. For **Ollama** backends, the count is a tiktoken-based estimate.

```
curl -X POST http://localhost:8000/v1/tokenize \
-H "Authorization: Bearer mr2_your-api-key" \
-H "Content-Type: application/json" \
-d '{
  "model": "openai/gpt-oss-120b",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello, how are you?"}
  ],
  "tools": [...]
}'
```

Response:

```
{
  "count": 847,
  "max_model_len": 131072,
  "is_estimate": false
}
```

Fields:

- `count` — number of input tokens (includes system prompt, messages, chat template overhead, and tool definitions)
- `max_model_len` — model's context window size
- `is_estimate` — `false` for vLLM (exact count), `true` for Ollama (tiktoken estimate)

Context Window Management

MindRouter automatically caps `max_tokens` before dispatching each request to prevent context overflow errors. The logic is:

```
input_tokens = tokenize(request)          # exact for vLLM, estimated for Ollama
remaining    = context_length - input_tokens - 1024 # 1024-token safety buffer
max_tokens   = min(requested_max_tokens, remaining, 65536)
```

- vLLM backends:** MindRouter calls the backend's `/tokenize` endpoint to get the exact input token count, including chat template inflation (e.g. Hermes tool-calling templates can add thousands of tokens). If the `/tokenize` call fails, it falls back to tiktoken estimation.
- Ollama backends:** Uses tiktoken (`cl100k_base`) for estimation. The 1024-token buffer absorbs the estimation error and chat template overhead. Ollama also auto-adjusts context internally, so overflow errors are rare.
- 65,536 hard cap:** No single response can exceed 65,536 output tokens, regardless of context window size.
- Context overflow:** If input tokens alone exceed the context window (minus buffer), MindRouter allows a minimal 256-token output. The backend may still reject the request if the input is genuinely too large for the model.

List Models

```
curl http://localhost:8000/v1/models \
-H "Authorization: Bearer mr2_your-api-key"
```

Response:

```
{
  "object": "list",
  "data": [
    {
      "id": "llama3.3:70b",
      "object": "model",
      "created": 1700000000,
      "owned_by": "mindrouter",
      "capabilities": {
        "multimodal": false,
        "embeddings": false,
        "structured_output": true,
        "thinking": false
      },
      "backends": ["node1-gpu0", "node1-gpu2"],
      "context_length": 32768,
      "model_max_context": 131072,
      "parameter_count": "70B",
      "quantization": "Q4_K_M",
      "family": "llama"
    }
  ]
}
```

Fields include:

- `context_length` — effective context window (num_ctx injected per request)
- `model_max_context` — architectural maximum context the model supports
- `parameter_count` — model size (e.g. "7B", "70B")
- `quantization` — quantization level (e.g. "Q4_K_M", "FP16")
- `family` — model family (e.g. "llama", "qwen2")
- `capabilities.thinking` — whether the model supports thinking/reasoning mode

Ollama-Compatible Endpoints

These endpoints accept and return data in Ollama's native format. Ollama clients can be pointed at MindRouter as a drop-in replacement.

Method	Path	Auth	Description
POST	<code>/api/chat</code>	API Key	Ollama chat (streaming by default)
POST	<code>/api/generate</code>	API Key	Ollama text generation
POST	<code>/api/embeddings</code>	API Key	Ollama embeddings
GET	<code>/api/tags</code>	API Key	List models (Ollama format)

List Models (Ollama Format)

```
curl http://localhost:8000/api/tags \
```

```
-H "Authorization: Bearer mr2_your-api-key"
```

Response:

```
{
  "models": [
    {
      "name": "llama3.3:70b",
      "model": "llama3.3:70b",
      "modified_at": "2026-02-28T12:00:00",
      "details": {
        "parent_model": "",
        "format": "gguf",
        "family": "llama",
        "parameter_size": "70B",
        "quantization_level": "Q4_K_M"
      },
      "context_length": 32768,
      "model_max_context": 131072
    }
  ]
}
```

Ollama Chat

```
curl -X POST http://localhost:8000/api/chat \
-H "Authorization: Bearer mr2_your-api-key" \
-H "Content-Type: application/json" \
-d '{
  "model": "llama3.2",
  "messages": [{"role": "user", "content": "Hello!"}],
  "stream": false
}'
```

Note: Ollama defaults to `stream: true`. Set `"stream": false` explicitly for non-streaming responses.

Thinking/Reasoning Mode (Ollama):

Use the `think` field at the top level:

```
// Qwen-style: boolean toggle
{"model": "qwen3-32k:32b", "messages": [...], "think": true, "stream": false}

// gpt-oss: string effort level
{"model": "gpt-oss-32k:120b", "messages": [...], "think": "high", "stream": false}
```

The response includes a `thinking` field in the message. For `/api/generate`, thinking content appears as a top-level `thinking` field alongside `response`.

Ollama Generate

```
curl -X POST http://localhost:8000/api/generate \
-H "Authorization: Bearer mr2_your-api-key" \
-H "Content-Type: application/json" \
-d '{"model": "llama3.2", "prompt": "Why is the sky blue?"}'
```

Anthropic-Compatible Endpoint

This endpoint accepts and returns data in the Anthropic Messages API format. Anthropic SDK clients (Python, TypeScript) can be pointed at MindRouter by setting `base_url`.

Method	Path	Auth	Description
POST	<code>/anthropic/v1/messages</code>	API Key	Anthropic Messages API (streaming and non-streaming)

Messages

```
curl -X POST http://localhost:8000/anthropic/v1/messages \
-H "Authorization: Bearer mr2_your-api-key" \
-H "Content-Type: application/json" \
-d '{
  "model": "llama3.2",
  "max_tokens": 500,
  "messages": [{"role": "user", "content": "Hello!"}]
}'
```

Response:

```
{
  "id": "msg_abc123...",
  "type": "message",
  "role": "assistant",
  "model": "llama3.2",
  "content": [
    {"type": "text", "text": "Hello! How can I help you today?"}
  ],
  "stop_reason": "end_turn",
  "stop_sequence": null,
  "usage": {
    "input_tokens": 10,
    "output_tokens": 12
  }
}
```

Streaming — Set `"stream": true` to receive Anthropic SSE events (`message_start`, `content_block_delta`, `message_stop`, etc.):

```
curl -X POST http://localhost:8000/anthropic/v1/messages \
-H "Authorization: Bearer mr2_your-api-key" \
-H "Content-Type: application/json" \
-d '{"model": "llama3.2", "max_tokens": 500, "messages": [{"role": "user", "content": "Hi"}], "stream": true}'
```

System Prompt:

```
{
  "model": "llama3.2",
  "max_tokens": 500,
  "system": "You are a helpful assistant.",
  "messages": [{"role": "user", "content": "Hello!"}]
}
```

SDK Usage (Python)

```
import anthropic
client = anthropic.Anthropic(
    base_url="http://localhost:8000/anthropic",
    api_key="mr2_your-api-key",
)
message = client.messages.create(
    model="llama3.2",
    max_tokens=500,
    messages=[{"role": "user", "content": "Hello!"}],
)
```

Supported features:

- System prompts (string or content block array)
- Multimodal inputs (base64 and URL images)
- Tool calling — `tools` with `input_schema`, `tool_choice` (auto/any/tool), `tool_use/tool_result` content blocks, streaming tool use with `input_json_delta`
- Thinking/reasoning mode (`thinking.type`: enabled, adaptive, disabled)
- Structured output via `output_config.format` with `type`: "json_schema"
- Parameters: `max_tokens`, `temperature`, `top_p`, `top_k`, `stop_sequences`, `stream`
- `metadata.user_id` mapping

Note: This is inbound-only — there are no Anthropic backends. Requests are translated to canonical format and routed to Ollama/vLLM backends like any other request.

Health & Metrics Endpoints

These endpoints are unauthenticated and intended for monitoring infrastructure.

Method	Path	Auth	Description
GET	/healthz	None	Liveness probe (always 200 if app is running)
GET	/readyz	None	Readiness probe (checks DB + healthy backends)
GET	/metrics	None	Prometheus metrics (text format)
GET	/status	None	Cluster status summary (JSON)
GET	/api/cluster/total-tokens	None	Total tokens ever served (cached 10s)
GET	/api/cluster/trends	None	Token and active-user trends (?range=hour day week month year)
GET	/api/cluster/throughput	None	Token throughput, requests/min, active requests

Prometheus Metrics

The `/metrics` endpoint exposes the following Prometheus metrics:

Metric	Type	Labels	Description
mindrouter_requests_total	Counter	endpoint, status	Total requests processed
mindrouter_request_latency_seconds	Histogram	endpoint	Request latency
mindrouter_queue_size	Gauge	—	Current scheduler queue depth
mindrouter_active_backends	Gauge	—	Number of healthy backends
mindrouter_tokens_total	Counter	type (prompt/completion)	Total tokens processed

Admin API Endpoints

Admin endpoints are mounted under `/api/admin/`. **Read-only** (GET) endpoints require membership in a group with `is_admin = true` or `is_auditor = true`. **Mutating** (POST/PATCH/DELETE) endpoints require `is_admin = true`. See [Users, Groups & Quotas](#) for details on the auditor role.

Backend Management

Method	Path	Description
POST	<code>/api/admin/backends/register</code>	Register a new inference backend
GET	<code>/api/admin/backends</code>	List all backends
PATCH	<code>/api/admin/backends/{id}</code>	Update backend properties
POST	<code>/api/admin/backends/{id}/disable</code>	Disable a backend
POST	<code>/api/admin/backends/{id}/enable</code>	Enable a disabled backend
POST	<code>/admin/backends/{id}/drain</code>	Initiate graceful drain (stops new requests, waits for in-flight to complete, then disables). Dashboard route only, not available via API.
POST	<code>/api/admin/backends/{id}/refresh</code>	Force-refresh capabilities and models
POST	<code>/api/admin/backends/{id}/ollama/pull</code>	Start pulling a model to an Ollama backend
GET	<code>/api/admin/backends/{id}/ollama/pull/{job_id}</code>	Poll progress of an Ollama model pull
POST	<code>/api/admin/backends/{id}/ollama/delete</code>	Delete a model from an Ollama backend

Node Management

Method	Path	Description
POST	<code>/api/admin/nodes/register</code>	Register a new GPU node
GET	<code>/api/admin/nodes</code>	List all nodes
PATCH	<code>/api/admin/nodes/{id}</code>	Update node properties
DELETE	<code>/api/admin/nodes/{id}</code>	Delete a node (fails if backends reference it)
POST	<code>/api/admin/nodes/{id}/refresh</code>	Force-refresh sidecar data

User Management

Method	Path	Description
GET	<code>/api/admin/users</code>	List users (filterable by group, searchable by name/email)
POST	<code>/api/admin/users</code>	Create a new user with group-based quota defaults
GET	<code>/api/admin/users/{id}</code>	User detail with usage stats, API keys, monthly usage
PATCH	<code>/api/admin/users/{id}</code>	Update user profile, group, and quota overrides
DELETE	<code>/api/admin/users/{id}</code>	Hard-delete a user and all associated data
POST	<code>/api/admin/users/{id}/api-keys</code>	Create an API key for a user

Group Management

Method	Path	Description
GET	/api/admin/groups	List all groups with user counts
POST	/api/admin/groups	Create a new group with quota defaults
PATCH	/api/admin/groups/{id}	Update group defaults (token budget, RPM, weight, etc.)
DELETE	/api/admin/groups/{id}	Delete a group (fails if users are assigned)

API Key Management

Method	Path	Description
GET	/api/admin/api-keys	List all API keys across users (searchable, filterable by status)

Quota Management

Method	Path	Description
GET	/api/admin/quota-requests	List pending quota increase requests
POST	/api/admin/quota-requests/{id}/review	Approve or deny a quota request

Conversations

Method	Path	Description
GET	/admin/conversations/export	Export conversations as CSV or JSON (form-based, filterable)
GET	/api/admin/conversations/export	Bulk API export (JSON with content, for programmatic access)

Queue & Audit

Method	Path	Description
GET	/api/admin/queue	Scheduler queue statistics
GET	/api/admin/audit/search	Search audit logs (filter by user, model, status, date, text)
GET	/api/admin/audit/{id}	Full audit detail including prompt and response content

Telemetry

Method	Path	Description
GET	/api/admin/telemetry/overview	Cluster-wide telemetry (nodes, backends, GPUs)
GET	/api/admin/telemetry/latest	Lightweight polling endpoint for dashboard
GET	/api/admin/telemetry/backends/{id}/history	Time-series telemetry for a backend
GET	/api/admin/telemetry/gpus/{id}/history	Time-series telemetry for a GPU device
GET	/api/admin/telemetry/nodes/{id}/history	Aggregated time-series for a node (all GPUs)
GET	/api/admin/telemetry/export	Export raw telemetry as JSON or CSV

Web Dashboard

MindRouter includes a full web dashboard built with Bootstrap 5. All pages extend a common base template with

navigation and accessibility features (WCAG 2.1 Level AA).

Public Pages

Page	URL	Description
Cluster Status	/	Shows healthy backend count, available models, queue size, and overall cluster status
Login	/login	Username/password authentication (Azure AD SSO when configured)
Blog	/blog	Public blog listing with published posts
Blog Post	/blog/{slug}	Individual blog post rendered from Markdown

User Dashboard

Page	URL	Description
Dashboard	/dashboard	Token usage progress bar, active API keys, quota usage history, change password
Change Password	POST /dashboard/change-password	Change password for local (non-SSO) accounts. Requires current password, new password (min 8 chars), and confirmation. Not available for Azure AD SSO users.
Request Quota	/dashboard/request-quota	Submit a quota increase request with justification
Key Created	(after creation)	Displays the full API key once (copy-to-clipboard)

The user dashboard includes:

- **Dark mode toggle** — saved to browser localStorage, persists across sessions
- **Live token usage** — usage statistics poll every 1 second for real-time feedback without page refresh
- **Lifetime vs rolling usage** — displays both *Lifetime Token Usage* (all-time total, never resets) and *Current Period Usage* (resets when the budget period rolls over)
- **Quota details** — current RPM limit shown in the Quota Details card
- **API key expiration warnings** — keys within 7 days of expiration show a yellow countdown; expired keys display an “Expired” badge; “Last Used” column shows last authentication time

Admin Dashboard

The admin dashboard has a persistent sidebar with links to all admin pages. Access requires membership in a group with `is_admin = true` or `is_auditor = true`. Auditors see all admin pages in read-only mode — mutating controls (create, edit, delete, toggle) are hidden.

Page	URL	Description
Overview	/admin	System metrics overview, health alert banner for unhealthy backends/offline nodes, pending request badges, system force offline/online toggle
Backends	/admin/backends	Backend health, models, enable/disable/drain controls
Nodes	/admin/nodes	GPU node management, sidecar status, hardware specs, take offline/bring online/force drain controls
Models	/admin/models	Model management: multimodal and thinking overrides, full metadata overrides (family, parameters, quantization, context length, etc.), Ollama pull/delete

Page	URL	Description
GPU Metrics	/admin/metrics	Real-time GPU utilization, memory, temperature, power charts with time range controls
Users	/admin/users	User accounts with search, group filter, and pagination
User Detail	/admin/users/{id}	Individual user profile, usage stats, API keys, monthly usage chart, quota overrides, masquerade (view dashboard as user)
Groups	/admin/groups	Group management: create, edit, delete groups with quota defaults and scheduler weights
API Keys	/admin/api-keys	All API keys across users with search, status filter, and pagination
Requests	/admin/requests	Pending API key and quota increase requests, approve/deny
Audit Log	/admin/audit	Inference request history with filtering, search, and CSV/JSON export
Admin Audit	/admin/admin-audit	Persistent log of all admin actions with before/after snapshots, filterable by action, entity type, and admin user
Conversations	/admin/conversations	Browse and export all user conversations
Chat Configuration	/admin/chat-config	Configure chat UI defaults: core models, default model, system prompt, max tokens, temperature, thinking mode
Search	/admin/search-config	Configure web search API: provider selection (Brave, SearXNG), API keys, quota cost, health checks, test search
Blog Management	/admin/blog	Blog CMS: create, edit, publish/unpublish, and delete blog posts
Site Settings	/admin/settings	Global settings: display timezone, Ollama context length enforcement

System Force Offline/Online

Administrators can force the entire MindRouter system offline from the admin overview page (</admin>). This is useful for planned maintenance windows or emergency situations.

- **Force Offline** (POST </admin/system/toggle-online>) — Stops the backend polling loop, marks all backends as unhealthy in the database, and sets an internal `_force_offline` flag. While offline, no health checks or discovery runs, and no inference requests can be served.
- **Force Online** — Clears the offline flag, closes and reloads all backend adapters and sidecar clients from the database, restarts the polling loop, and immediately runs a full poll cycle to restore backend health status.

The toggle is available as a button on the admin overview page. The system status is visible to all users on the public status page.

Node Lifecycle Management

Beyond basic node registration and editing, administrators can manage node operational state from the Nodes page (</admin/nodes>):

Action	Route	Description
Take Offline	POST /admin/nodes/{id}/take-offline	Disables all backends on the node and marks the node status as OFFLINE. New requests will not be routed to any backend on this node.

Action	Route	Description
Bring Online	<code>POST /admin/nodes/{id}/bring-online</code>	Re-enables all backends on the node and marks the node status as ONLINE. Backends will begin receiving requests again after the next health poll.
View Active Requests	<code>GET /admin/nodes/{id}/active-requests</code>	Returns a JSON count of in-flight requests across all backends on the node. Useful for monitoring drain progress before taking a node offline.
Force Drain	<code>POST /admin/nodes/{id}/force-drain</code>	Force-cancels all active (in-flight) requests on the node's backends. Returns the number of cancelled requests. Use this when you need to take a node offline immediately without waiting for requests to complete naturally.

Recommended workflow for node maintenance:

1. Take the node offline to stop new requests from being routed there.
2. Monitor active requests using the active requests endpoint until the count reaches zero.
3. If requests are stuck, use force drain to cancel them.
4. Perform maintenance (upgrade vLLM, restart Ollama, etc.).
5. Bring the node back online.

Admin Models Page

The Models page at `/admin/models` provides comprehensive model management. Models are grouped by name (since the same model may appear on multiple backends) and presented with their current metadata and override status.

Capability Overrides

MindRouter auto-detects model capabilities during discovery, but admins can override the detected values:

- **Multimodal override** — Toggle or reset the `supports_multimodal` flag for all instances of a model. Use this when auto-detection incorrectly classifies a model (e.g., a vision model whose name does not match the usual patterns).
- **Thinking override** — Toggle or reset the `supports_thinking` flag for all instances. Use this for models that support thinking/reasoning mode but are not detected automatically.
- Resetting an override returns the field to auto-detected values from the next discovery cycle.

Metadata Overrides

Admins can override any model metadata field across all instances of a model via the metadata edit form. Overridable fields:

Field	Type	Description
<code>family</code>	string	Model family (e.g., <code>llama</code> , <code>qwen2</code>)
<code>parameter_count</code>	string	Model size (e.g., <code>7B</code> , <code>70B</code>)
<code>quantization</code>	string	Quantization level (e.g., <code>Q4_K_M</code> , <code>FP16</code>)
<code>context_length</code>	int	Effective context window (injected as <code>num_ctx</code>). Overrides the auto-discovered value.
<code>embedding_length</code>	int	Embedding dimension length
<code>head_count</code>	int	Number of attention heads
<code>layer_count</code>	int	Number of transformer layers
<code>feed_forward_length</code>	int	Feed-forward network dimension
<code>model_format</code>	string	Model format (e.g., <code>gguf</code> , <code>safetensors</code>)

Field	Type	Description
parent_model	string	Parent model identifier
capabilities	string	Comma-separated list of capabilities (stored as JSON array)
description	string	Human-readable model description
model_url	string	URL to model documentation or repository

Clearing a field (setting it to blank) removes the override and reverts to the auto-discovered value. Admins can also reset all overrides at once via the "Reset All Overrides" action.

Ollama Model Pull and Delete

The Models page lists all Ollama backends and provides UI controls to:

- **Pull a model** — Trigger a model download on a specific Ollama backend via the admin API (POST `/api/admin/backends/{id}/ollama/pull`). Progress can be polled until completion.
- **Delete a model** — Remove a model from a specific Ollama backend via the admin API (POST `/api/admin/backends/{id}/ollama/delete`).

Model Auto-Enrichment

MindRouter can automatically generate structured descriptions for models that have no description. This runs as a background task that searches the web for model cards and summarizes them using MindRouter's own inference API.

How It Works

1. Every 120 seconds, the enrichment loop checks for models with no description (up to 3 per cycle).
2. For each model, it searches the web via the **Brave Search API** for the model's card or documentation page.
3. It then calls MindRouter's own `/v1/chat/completions` endpoint with the search results and model metadata, asking an LLM to produce a structured markdown description.
4. The generated description is saved to **all instances** of that model across backends.

There is a 90-second startup delay before the first enrichment cycle, giving backends time to be discovered. A 5-second delay between individual model enrichments provides rate limiting.

Configuration

Auto-enrichment is configured via **Admin > Settings**. Four [AppConfig](#) keys control the feature:

Key	Type	Default	Description
catalog.auto_enrich	bool	false	Master toggle — enable or disable auto-enrichment
catalog.enrich_model	string	""	Model name to use for generating descriptions (e.g. <code>qwen3.5:32b</code>). Must be available in the cluster.
catalog.enrich_api_key	string	""	API key for internal MindRouter inference calls
catalog.brave_api_key	string	""	Brave Search API key. Overrides the <code>BRAVE_SEARCH_API_KEY</code> environment variable if set.

All four values must be configured for enrichment to run. The enrichment model and API key fields are required; if either is empty, the loop exits immediately.

Behavior Details

- **Only NULL descriptions are enriched** — models with any existing description (including admin-written ones) are never overwritten.

- **Re-enrichment** — To re-generate a description, clear it via the Admin Models page. The next enrichment cycle will pick it up.
- **Batch size** — At most 3 models are enriched per 120-second cycle to avoid flooding backends with enrichment requests.
- **Deduplication** — The same model name is only processed once per batch, even if it appears on multiple backends. The resulting description is applied to all instances.
- **Failures are non-fatal** — If a web search or LLM call fails for a specific model, it is logged and skipped. The loop continues with the next model and will retry the failed model in a subsequent cycle.
- **Description cache** — Generated descriptions are cached in a separate `model_description_cache` table keyed by model name. When a model row is deleted (e.g. by `remove_stale_models` during node maintenance) and later rediscovered, the enrichment loop restores the description instantly from the cache instead of repeating the web search and LLM call. Cache hits are logged as `model_enriched_from_cache`.

Admin Masquerade

Administrators can "masquerade" as another user to view the user dashboard from that user's perspective. This is useful for troubleshooting user-reported issues or verifying quota and usage display.

- **Start masquerade** (POST `/admin/masquerade/{target_user_id}`) — Sets a signed cookie (`mindrouter_masquerade`) containing the target user ID. The cookie is signed using `itsdangerous.URLSafeTimedSerializer` with the application's `SECRET_KEY` and a "masquerade" salt. The cookie expires after 24 hours.
- **During masquerade** — The user dashboard (`/dashboard`) shows the target user's data (usage stats, API keys, quota, conversations) instead of the admin's own data. The masquerade applies only to read-only dashboard views — admin routes and actions are never affected.
- **Stop masquerade** (POST `/admin/masquerade/stop`) — Deletes the masquerade cookie and redirects back to the admin users page.
- **Security** — Only users in an admin group can start a masquerade. The target user ID is verified to exist before the cookie is set. The signed cookie prevents tampering.

Masquerade is initiated from the user detail page (`/admin/users/{id}`) via a "View as User" button.

Audit Log Export

Administrators can export audit log data as CSV or JSON via GET `/admin/audit/export`. The export supports the same filters as the audit log UI:

Parameter	Type	Description
<code>format</code>	string	Output format: <code>csv</code> (default) or <code>json</code>
<code>search</code>	string	Free-text search across request fields
<code>user_id_filter</code>	int	Filter by user ID
<code>model_filter</code>	string	Filter by model name
<code>status_filter</code>	string	Filter by request status
<code>start_date</code>	string	Start date (ISO 8601 format)
<code>end_date</code>	string	End date (ISO 8601 format)
<code>include_content</code>	bool	Include prompt/response content in export (default: false)

Export fields: `request_uuid`, `created_at`, `user_id`, `model`, `endpoint`, `status`, `prompt_tokens`, `completion_tokens`, `total_tokens`, `total_time_ms`, `error_message`. When `include_content=true`, additional fields are included: `messages`,

`prompt`, `parameters`, `response_content`, `finish_reason`.

Exports are limited to 10,000 records. The file is downloaded as an attachment (`audit_log.csv` or `audit_log.json`).

Admin Audit Log

Every mutating admin action — whether triggered via the REST API (`/api/admin/...`) or the dashboard UI — is recorded in the `admin_audit_log` database table. This provides a persistent, tamper-evident record of all administrative changes to the system.

What Gets Recorded

Field	Type	Description
<code>timestamp</code>	<code>datetime</code>	Auto-set by the database when the action occurs
<code>user_id</code>	<code>int (FK)</code>	The admin user who performed the action
<code>action</code>	<code>string</code>	Verb describing the action: <code>create</code> , <code>update</code> , <code>delete</code> , <code>enable</code> , <code>disable</code> , <code>toggle_tools</code> , <code>toggle_multimodal</code> , <code>toggle_thinking</code> , <code>drain</code> , <code>pull_model</code> , <code>delete_model</code> , <code>approve</code> , <code>deny</code> , etc.
<code>entity_type</code>	<code>string</code>	What was acted on: <code>backend</code> , <code>model</code> , <code>node</code> , <code>user</code> , <code>group</code> , <code>api_key</code> , <code>setting</code> , <code>blog_post</code> , <code>quota_request</code> , <code>system</code> , etc.
<code>entity_id</code>	<code>string</code>	ID or name of the specific entity (e.g., backend ID, model name, user ID)
<code>before_value</code>	<code>JSON</code>	Snapshot of the entity's state before the change (when applicable)
<code>after_value</code>	<code>JSON</code>	Snapshot of the entity's state after the change
<code>ip_address</code>	<code>string</code>	Client IP address of the admin who made the request
<code>detail</code>	<code>text</code>	Human-readable summary of the action (e.g., "Disabled backend vllm-aspen1")

Coverage

Admin audit logging is instrumented across two entry points:

- **Admin API** (`/api/admin/...`) — All mutating endpoints (`create`, `update`, `delete`, `enable`, `disable`, `drain`, `register`, `refresh`, etc.) log actions automatically.
- **Dashboard UI** (`/admin/...`) — All admin form submissions and action buttons (`toggle controls`, `pull/delete models`, `approve/deny requests`, `update settings`, etc.) log actions automatically.

Before/after JSON snapshots capture the entity state around each mutation. Create actions record only `after_value`, delete actions record only `before_value`, and updates capture both.

Viewer

The Admin Audit page at `/admin/admin-audit` provides a paginated, filterable table of all admin actions. Filters include:

- **Action** — Filter by action type (e.g., only show `delete` actions)
- **Entity Type** — Filter by entity type (e.g., only show `backend` changes)
- **Admin User** — Filter by which admin performed the action

Clicking any row expands it to reveal the full before/after JSON diff for that action.

Admin Chat Configuration

The Chat Configuration page at `/admin/chat-config` allows administrators to control default settings for the built-in chat interface. All settings are stored in the [AppConfig](#) database and take effect immediately.

Setting	Description	Details
Core Models	Subset of models shown in the chat model selector	Select from all available (non-embedding) models on healthy backends. When set, only these models appear in the chat UI dropdown. Empty means show all models.
Default Model	Pre-selected model in the chat UI	The model automatically selected when a user starts a new conversation. Blank means no default.
System Prompt	Global system prompt for all chat conversations	Prepended to every chat conversation. Blank removes the override and reverts to the built-in default. Can also be explicitly reset via a "Reset" button.
Max Tokens	Default max tokens for chat requests	Range: 256–131072. Default: 16384. Clamped to the allowed range on save.
Temperature	Default temperature for chat responses	Range: 0.0–2.0. Blank means use the model's default temperature.
Thinking Mode	Default thinking/reasoning mode	Options: <code>true</code> , <code>false</code> , <code>low</code> , <code>medium</code> , <code>high</code> , or blank (no default). Controls whether thinking is enabled by default for models that support it.

Chat Interface

Page	URL	Description
Chat	/chat	Full-featured chat UI with model selection, streaming, file upload, multimodal support

The chat interface supports:

- Collapsible conversation sidebar
- Model and backend selection
- Real-time streaming responses
- File upload via button or drag-and-drop anywhere in the chat window (images, PDFs, DOCX, XLSX, CSV, JSON, Markdown, etc.)
- Vision model support with automatic image handling
- Web search toggle — when enabled, queries the Brave Search API and injects results as context into the system prompt (requires `BRAVE_SEARCH_API_KEY`)
- Code syntax highlighting
- LaTeX rendering
- Message editing and deletion
- Dark mode toggle (stored in browser localStorage)
- Advanced models toggle — show or hide non-core models in the model selector
- Per-request thinking controls — enable/disable thinking mode and set reasoning effort for supported models
- Collapsible thinking blocks in assistant responses
- Keyboard shortcuts — Shift+Enter for newline, Enter to send
- Sidebar collapse for a wider chat area
- Copy buttons on code blocks and assistant messages
- Image lightbox for viewing uploaded images full-size
- Auto-titling of conversations based on the first message

Users, Groups & Quotas

Group System

MindRouter uses a **database-driven group system** for authorization, quota defaults, and scheduler weights. Each user belongs to exactly one group. Groups are fully manageable via the admin UI at `/admin/groups`.

Each group has two privilege flags:

- **is_admin** — Full admin access. Users can view all admin pages and perform all administrative actions (create, edit, delete users/groups/backends/nodes, approve quotas, change settings, etc.).
- **is_auditor** — Read-only admin access. Users can view all admin pages (dashboard, users, backends, nodes, metrics, audit logs, conversations, settings, etc.) but cannot make any changes. Mutating controls are hidden in the dashboard, and mutating API endpoints return HTTP 403.

A group can have `is_admin`, `is_auditor`, or neither. Admin groups implicitly have auditor-level read access, so setting both flags is unnecessary (though harmless). The auditor role is intended for oversight, compliance review, or stakeholders who need visibility into system operations without the ability to modify state.

Default Groups

Group	Token Budget	RPM	Sched. Weight	Role
students	100,000	30	1	—
staff	500,000	60	2	—
faculty	1,000,000	120	3	—
researchers	1,000,000	120	3	—
admin	10,000,000	1,000	10	Admin
auditor	100,000	30	1	Auditor
nerds	500,000	60	2	—
other	100,000	30	1	—

These 8 groups are created automatically by the database migration. Admins can create additional groups, edit defaults, or delete empty groups via the admin UI or API.

Auditor Role Details

The auditor role provides complete read-only visibility into MindRouter administration. This is useful for:

- **Compliance & oversight** — Auditors can review user accounts, API key usage, request audit logs, admin audit trails, and system configuration without risk of accidental changes.
- **Stakeholder visibility** — Give managers or department leads a view into cluster health, usage metrics, and capacity without granting mutation rights.
- **Security review** — Reviewers can inspect all admin pages and data to verify policies are being followed.

What auditors can access

- All admin dashboard pages (overview, backends, nodes, models, GPU metrics, energy, users, user detail, groups, API keys, requests, audit log, admin audit, conversations, chat config, voice config, settings, retention, backup)
- All read-only (GET) admin API endpoints
- Masquerade (view a user's dashboard as them) for investigation purposes
- Export features (audit CSV/JSON, conversation export, backup export)

What auditors cannot do

- Create, edit, or delete users, groups, backends, or nodes
- Enable, disable, or drain backends
- Approve or deny quota requests
- Pull or delete Ollama models
- Change system settings, retention policies, or chat/voice configuration
- Toggle the system online/offline
- Send emails or manage blog posts
- Restore backups

Setting up auditors

To grant auditor access, either assign users to the built-in `auditor` group, or create a custom group with `is_auditor = true`. Via the API:

```
# Create a custom auditor group
curl -X POST https://mindrouter.example.edu/api/admin/groups \
  -H "X-API-Key: YOUR_ADMIN_KEY" \
  -H "Content-Type: application/json" \
  -d '{"name": "security-reviewers", "display_name": "Security Reviewers", "is_auditor": true}'

# Move a user to the auditor group
curl -X PATCH https://mindrouter.example.edu/api/admin/users/42 \
  -H "X-API-Key: YOUR_ADMIN_KEY" \
  -H "Content-Type: application/json" \
  -d '{"group_id": 8}'
```

User Profiles

Each user has the following profile fields (editable by admins at `/admin/users/{id}`):

- **Username** and **Email** — unique identifiers
- **Full Name** — display name
- **Group** — determines default quotas, scheduler weight, and admin access
- **College** and **Department** — organizational affiliation
- **Intended Use** — free-text description of how the user plans to use the service

Quota Inheritance & Overrides

When a user is created, their quota defaults are inherited from their group. Admins can override any quota value per-user:

- **Token budget** — inherited from group, overridable per user
- **RPM limit** — inherited from group, overridable per user or per API key
- **Weight override** — if set, overrides the group's scheduler weight for this user

API Key Lifecycle

1. **Generation** — Keys use the format `mr2_<random_urlsafe_base64>` (48+ characters total).
2. **Storage** — The raw key is shown once at creation. Only the Argon2 hash and a prefix (`mr2_<first 8 chars>`) are stored in the database.
3. **Verification** — Lookup by prefix (fast), then full Argon2 hash verification.
4. **Expiration** — Optional `expires_at` timestamp.
5. **Revocation** — Keys can be revoked (soft-delete) without deleting the audit trail.
6. **Usage tracking** — `last_used_at` and `usage_count` updated atomically on each request.

Quota System

Each user has a quota record with:

- **Token budget** — Total tokens allowed per period. Deducted on each completed request (prompt + completion tokens).
- **RPM limit** — Maximum requests per minute. Enforced cluster-wide via Redis (shared across all workers). API key overrides take precedence over the user's quota RPM.

When a quota is exceeded, the request is rejected with HTTP 429. Token budget is checked first, then RPM. If Redis is unavailable, RPM enforcement fails open (requests are allowed).

Rolling Budget Period

Token budgets use **rolling 30-day windows** per user, not calendar months. Usage is tracked continuously and the oldest usage falls off as the window advances.

Quota Increase Requests

Users can submit quota increase requests via the dashboard (</dashboard/request-quota>). The request includes:

- Desired token budget
- Written justification

Admins review requests at </admin/requests> or via POST </api/admin/quota-requests/{id}/review>, which can approve (with a custom granted token amount) or deny the request.

Service API Keys

What Are Service Keys?

Service API Keys are production-grade keys intended for long-running services, automated pipelines, and integrations that require uninterrupted API access. Unlike regular personal API keys, service keys:

- **Never expire** — no need to rotate keys on a schedule or worry about service disruptions from key expiration.
- **Are admin-managed** — only administrators can approve, promote, and revoke service keys.
- **Carry metadata** — each service key tracks the service name, data risk level, compliance tags, and alternative contacts.

How to Request a Service Key

1. Go to your **Dashboard** and find the API key you want to promote.
2. Click **Request Service Key** next to any active personal key.
3. Fill in the required information:
 - **Service / Application Name** — the name of the service or pipeline that will use this key.
 - **Justification** — why this service needs a non-expiring key.
 - **Alternative Contacts** — emails of people who can manage this key if you are unavailable.
 - **Data Risk Level** — Low, Medium, or High based on the sensitivity of data processed.
 - **Compliance Tags** — applicable data classifications (FERPA, HIPAA, CUI, FCI, PII, IP, or Other).
4. Submit the request. An administrator will review it.

Approval Process

Administrators review service key requests from the **Admin → Requests** page. When approved:

- The key is promoted to service status (the underlying key value does not change).
- The expiration date is removed (the key never expires).
- Service metadata (name, contacts, risk level, compliance tags) is attached to the key.

If denied, the key remains a regular personal key with its original expiration.

Revocation

Service keys cannot be self-revoked by users. To revoke a service key:

1. The key owner submits a **Revocation Request** from the dashboard, providing a reason.
2. An administrator reviews the request and executes the revocation from the admin panel.

This two-step process prevents accidental revocation of keys that may be in active production use.

Backend Management

Node/Backend Model

MindRouter separates the concept of physical GPU servers (**Nodes**) from inference endpoints (**Backends**):

- A **Node** represents a physical server with GPUs and a sidecar agent.
- A **Backend** is an Ollama or vLLM instance running on a node.
- One node can host multiple backends, each assigned specific GPUs via `gpu_indices`.
- Backends without a `node_id` work as standalone endpoints (no GPU telemetry).

```
Node: gpu-server-1 (4x A100-80GB, sidecar at :8007)
+-- Backend: vllm-large (gpu_indices: [0, 1]) ← uses GPUs 0-1
+-- Backend: vllm-small (gpu_indices: [2])    ← uses GPU 2
+-- Backend: ollama-misc (gpu_indices: [3])   ← uses GPU 3
```

Supported Engines

Engine	Health Check	Model Discovery	Telemetry Source
Ollama	GET /api/tags	GET /api/tags + POST /api/ps (loaded models)	Sidecar agent
vLLM	GET /health (fallback: GET /v1/models)	GET /v1/models	GET /metrics (Prometheus format)

Registration

Register a node:

```
curl -X POST http://localhost:8000/api/admin/nodes/register \
-H "Authorization: Bearer admin-api-key" \
-H "Content-Type: application/json" \
-d '{
  "name": "gpu-server-1",
```

```

    "hostname": "gpu1.example.com",
    "sidecar_url": "http://gpu1.example.com:8007",
    "sidecar_key": "your-sidecar-secret-key"
  },

```

Register a backend on that node:

```

curl -X POST http://localhost:8000/api/admin/backends/register \
-H "Authorization: Bearer admin-api-key" \
-H "Content-Type: application/json" \
-d '{
  "name": "ollama-gpu1",
  "url": "http://gpu1.example.com:11434",
  "engine": "ollama",
  "max_concurrent": 4,
  "node_id": 1,
  "gpu_indices": [0, 1]
}'

```

Enable/Disable/Drain/Refresh

- **Disable** a backend to take it out of rotation without deleting it: `POST /api/admin/backends/{id}/disable`
- **Enable** to bring it back: `POST /api/admin/backends/{id}/enable`
- **Drain** for graceful offline maintenance: `POST /api/admin/backends/{id}/drain` (dashboard-only route)
- **Refresh** to force re-discovery of models and capabilities: `POST /api/admin/backends/{id}/refresh`

Drain Mode

Drain mode provides graceful backend shutdown for maintenance. When a backend is set to draining:

1. The scheduler stops routing **new** requests to the backend.
2. All **in-flight** requests are allowed to complete normally.
3. When the backend's queue depth reaches 0, it automatically transitions to **disabled**.

This avoids abruptly killing active requests when you need to restart an inference engine, upgrade models, or perform node maintenance. Use it before upgrading vLLM, restarting Ollama, or taking a GPU node offline.

Concurrency Alignment

The `max_concurrent` value registered in MindRouter **must match** the concurrency limit on the inference engine:

Engine	Engine Setting	MindRouter Setting
vLLM	<code>--max-num-seqs N</code>	<code>"max_concurrent": N</code>
Ollama	<code>OLLAMA_NUM_PARALLEL=N</code>	<code>"max_concurrent": N</code>

Why this matters: MindRouter uses `max_concurrent` to decide how many requests to route to a backend. If MindRouter thinks a backend can handle 8 concurrent requests but vLLM is configured with `--max-num-seqs 4`, the extra requests queue silently inside vLLM. MindRouter cannot see this hidden queue, so it continues routing requests there instead of spreading load to other backends. The result is uneven load distribution and unpredictable latency — the fair-share scheduler is effectively bypassed for those excess requests.

Context Length (num_ctx)

MindRouter automatically manages context length for Ollama backends:

- **Auto-discovery** — During model discovery, `context_length` is set to `min(model_max_context, 32768)` to prevent small models from consuming excessive VRAM. The architectural maximum is stored separately as `model_max_context`.
- **num_ctx injection** — For every Ollama request, MindRouter injects `num_ctx` matching the model's configured `context_length`.
- **Enforcement toggle** — By default, MindRouter overrides any user-supplied `num_ctx` to prevent GPU memory oversubscription. Admins can toggle this in Site Settings (`/admin/settings`) to allow users to set their own `num_ctx`.
- **Manual override** — Admins can set `context_length_override` per model via the admin UI to use a custom value instead of the auto-discovered one.
- **Ollama 0.17+** — Ollama automatically adjusts `num_ctx` downward if the requested value doesn't fit in GPU memory.

vLLM backends handle context length natively via `--max-model-len` and do not need `num_ctx` injection.

Scheduling & Fair Share

MindRouter implements **Weighted Deficit Round Robin (WDRR)** to ensure fair GPU access across users.

How It Works

1. **Share weights** are determined by the user's group (e.g., `students=1`, `staff=2`, `faculty=3`, `admin=10`). Per-user weight overrides are supported via the quota system.
2. Each user has a **deficit counter** that tracks how much service debt they've accrued.
3. On each scheduling round, users with the highest deficit (most underserved) are served first.
4. **Burst credits** allow full cluster utilization when the cluster is idle.
5. **Heavy user deprioritization** kicks in when a user exceeds their fair share within the fairness window.

Backend Scoring

When multiple backends can serve a request, the scheduler scores them on:

- **Model already loaded** (+100 points) — avoids cold-loading the model
- **Low GPU utilization** (+50 points) — prefers idle GPUs
- **Low latency** (+40 points) — based on EMA of recent response times
- **Short queue** (+30 points) — prefers backends with fewer queued requests
- **High throughput** (+20 points) — based on recent tokens/second
- **Priority** (+N × 10 points) — from the backend's configured `priority` value

Hard constraints (multimodal capability, embedding support, model availability) are checked before soft scoring.

Priority Gating

When multiple requests are waiting for the same model, the scheduler ensures the **highest-priority waiter proceeds first**. Priority is determined by the user's group scheduler weight and deficit counter, preventing lower-priority requests from starving higher-priority ones during contention.

Retry & Failover

MindRouter automatically retries failed inference requests to improve reliability across the backend cluster.

- **Automatic retries** — Up to 3 attempts on 5xx errors, timeouts, and connection failures. Configurable via the

`BACKEND_RETRY_MAX_ATTEMPTS` environment variable.

- **Fast fail on 4xx** — Client errors (400, 401, 404, etc.) are never retried and return immediately.
- **Streaming constraints** — Retries can only occur before the first chunk is sent to the client. Once streaming has begun, a backend failure is terminal and the stream ends with an error.
- **Backend rotation** — Each retry attempt selects a different backend when multiple backends are available for the requested model, maximizing the chance of success.
- **Clean timeout cancellation** — Each backend request uses a dedicated HTTP connection. When a request times out (default 180s per attempt), MindRouter closes the connection, which signals the backend (vLLM) to abort the in-progress generation and free the execution slot. This prevents orphaned requests from consuming backend capacity after MindRouter has given up.

Mid-Stream Error Behavior

Once streaming has begun and the first chunk has been sent to the client, a backend failure **terminates the SSE stream immediately** with no error event. The client receives an abrupt end-of-stream. Retries are not possible after the first chunk because the response format has already been committed.

Circuit Breaker

MindRouter uses a per-backend circuit breaker to avoid routing requests to backends that are experiencing persistent failures.

- **Trip threshold** — After 3 consecutive failures (configurable), the backend is marked as "open" and excluded from routing for 30 seconds (configurable).
- **Integration with retry** — The circuit breaker works alongside retry. Backends with an open circuit are skipped during failover selection, so retries are directed to healthier backends.
- **Automatic recovery** — After the exclusion window expires, the backend is eligible for routing again. A successful request resets the failure counter.

Rate Limiting

MindRouter enforces two types of rate limits. Both return HTTP 429 when exceeded — clients should implement exponential backoff.

- **Token quota** — Rolling token budget per user (period defaults to 30 days). When the budget is exhausted, all requests are rejected until the period resets. Configured per group, overridable per user.
- **Requests per minute (RPM)** — Enforced cluster-wide via Redis using an atomic INCR + EXPIRE counter with a 60-second sliding window. The limit is shared across all application workers, so a user configured for 120 RPM gets exactly 120 RPM regardless of which worker handles the request. Configured per group, overridable per user or per API key. If Redis is unavailable, RPM enforcement fails open (requests are allowed).

Tool Calling

MindRouter supports tool calling (function calling) across all three API formats, with transparent translation between them.

- **Tool definitions** — Pass an OpenAI-style `tools` array describing available functions, with optional `tool_choice`

("auto", "none", or a specific function name).

- **Tool results** — Submit results back via `role: "tool"` messages with a matching `tool_call_id`.
- **Cross-format support** — Tool calls work whether the request arrives in OpenAI, Ollama, or Anthropic format. MindRouter translates tool definitions, tool calls, and tool results between all formats automatically.
- **Backend requirement** — The selected backend must support tool calling. For vLLM, this requires `--enable-auto-tool-choice` and the appropriate `--tool-call-parser` flag. For Ollama, tool calling is supported natively on compatible models.

Document OCR

MindRouter provides a dedicated `POST /v1/ocr` endpoint that converts images, PDFs, and Office documents into structured markdown or JSON using a multimodal LLM. This is particularly useful for extracting text from scanned documents, research papers, forms, and spreadsheets while preserving layout, tables, and formatting.

Supported Formats

- **Images:** PNG, JPEG, WebP, GIF, TIFF, BMP (including multi-frame GIF/TIFF)
- **PDF:** Converted to page images at configurable DPI (default 200)
- **Office:** DOCX, PPTX, XLSX — converted to PDF via LibreOffice, then to images

How It Works

1. **Document to images** — The uploaded file is converted to a series of PNG page images.
2. **Chunking** — For multi-page documents, pages are split into overlapping chunks (default: 6 pages per chunk, 2-page overlap). This ensures no content is lost at page boundaries.
3. **Parallel OCR** — Each chunk is sent to a multimodal LLM as a chat completion request containing the page images and an OCR prompt. Chunks are processed concurrently for speed.
4. **Completeness check** — If a chunk's output is suspiciously short (below a configurable threshold per page), it is automatically retried with a stronger prompt.
5. **Deterministic merge** — Overlapping chunks are merged using `difflib` sequence matching to find the overlap boundary, then spliced together. No additional LLM calls are needed for merging.

API Usage

Send a multipart `POST` request with your API key and the file:

```
curl -X POST https://mindrouter.uidaho.edu/v1/ocr \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -F "file=@document.pdf" \
  -F "output_format=markdown"
```

Request Parameters (form fields)

Field	Type	Default	Description
<code>file</code>	file	<i>required</i>	The document to OCR (image, PDF, or Office file)
<code>model</code>	string	Admin-configured	Multimodal model to use (must support vision)
<code>output_format</code>	string	<code>markdown</code>	<code>markdown</code> or <code>json</code>
<code>chunk_size</code>	int	6	Pages per chunk for multi-page documents
<code>overlap</code>	int	2	Overlapping pages between chunks
<code>dpi</code>	int	200	Image resolution for PDF/Office conversion

Response

```
{
  "id": "ocr-abc123...",
  "object": "ocr.result",
  "created": 1711700000,
  "model": "qwen/qwen3.5-122b",
  "content": "# Document Title\n\n## Section 1\n\nExtracted text...",
  "format": "markdown",
  "pages": 12,
  "chunks_processed": 3,
  "usage": {
    "prompt_tokens": 45000,
    "completion_tokens": 8000,
    "total_tokens": 53000
  }
}
```

Python Example

```
import requests

response = requests.post(
    "https://mindrouter.uidaho.edu/v1/ocr",
    headers={"Authorization": "Bearer YOUR_API_KEY"},
    files={"file": open("paper.pdf", "rb")},
    data={"output_format": "markdown"},
)

result = response.json()
print(result["content"]) # Markdown text
print(f"Pages: {result['pages']}, Chunks: {result['chunks_processed']}")
```

Simplified Endpoint: `/v1/ocrmd`

For convenience, `POST /v1/ocrmd` returns **raw markdown** directly (not JSON-wrapped). The response body is plain text with `Content-Type: text/markdown`. This is useful for piping output directly to a file:

```
curl https://mindrouter.uidaho.edu/v1/ocrmd \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -F "file=@document.pdf" \
  > output.md
```

Accepts the same optional parameters as `/v1/ocr` (`model`, `chunk_size`, `overlap`, `dpi`) except `output_format` (always markdown).

Configuration

Administrators can configure OCR settings via **Admin → OCR** in the web dashboard. Configurable options include:

- **Default model** — Which multimodal model to use (must support vision/image inputs)
- **Chunk size & overlap** — How many pages per chunk and how many pages overlap between chunks
- **DPI** — Resolution for PDF and Office document conversion
- **Concurrency** — How many chunks to process in parallel
- **Safety limits** — Maximum pages, file size, retries per chunk
- **Temperature & max tokens** — LLM parameters for OCR requests
- **Enable/disable** — Toggle the entire OCR endpoint

All settings are stored in the `app_config` table and take effect immediately without restarting the application.

Web Search

MindRouter provides a standalone web search API at `POST /v1/search` (and `POST /api/search`). This is a first-class service — it does **not** modify or inject results into LLM calls. Clients decide how to use the results (feed them to an agent, display to a user, etc.).

Usage

Authenticate with your MindRouter API key. Each search deducts a small configurable token cost from your quota.

```
curl -X POST https://mindrouter.uidaho.edu/v1/search \
  -H "Authorization: Bearer $MINDROUTER_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"query": "latest advances in LLM inference", "max_results": 5}'
```

Request Body

Field	Type	Default	Description
<code>query</code>	string	<i>required</i>	The search query (1–2000 characters)
<code>max_results</code>	int	from config	Maximum results to return (1–50). If omitted, uses the server default.

Response

```
{
  "query": "latest advances in LLM inference",
  "provider": "Brave Search",
  "results": [
    {
      "title": "Advances in LLM Inference Optimization",
      "url": "https://example.com/article",
      "snippet": "Recent work on speculative decoding and ...",
      "published": "2026-04-10"
    }
  ],
  "total_results": 5
}
```

Python Example

```
import requests

resp = requests.post(
    "https://mindrouter.uidaho.edu/v1/search",
    headers={"Authorization": f"Bearer {api_key}"},
    json={"query": "transformer architecture improvements", "max_results": 10},
)
results = resp.json()["results"]
for r in results:
    print(f"{r['title']}: {r['url']}")
```


Pluggable Providers

The search backend is configurable via **Admin → Search**. MindRouter supports multiple search providers:

Provider	Type	API Key Required	Notes
Brave Search	Commercial API	Yes	Fast, high-quality web results. Free tier available at brave.com/search/api
SearXNG	Self-hosted	No	Meta-search engine aggregating multiple upstream providers. Requires a running SearXNG instance with JSON format enabled.

Agentic AI Integration

The `/v1/search` endpoint is available as both a pre-built [MCP server](#) and an [agent skill](#) for AI coding agents. Both integrations live in the `agentic_ai/` directory.

Configuration

All search settings are managed via **Admin → Search** in the web dashboard:

- **Enable/disable** — Toggle the entire search API (returns 503 when disabled)
- **Active provider** — Select which search backend to use
- **Default max results** — Server-wide default (clients can override up to 50)
- **Quota cost** — Fixed token cost deducted per search request (default 50)
- **Provider-specific settings** — API keys, endpoint URLs per provider
- **Health checks** — Real-time status of each configured provider
- **Test search** — Inline test with live results preview

Settings are stored in the `app_config` table (keys prefixed with `search.*`) and take effect immediately.

🔗 MCP Servers

MindRouter provides [Model Context Protocol \(MCP\)](#) servers that expose MindRouter APIs as tools for agentic systems like **Claude Code**, **CoWork**, **Cursor**, and other MCP-compatible clients.

Two connection options are available:

- **Hosted SSE server (recommended)** — Connect directly to MindRouter's built-in MCP endpoint. No local dependencies or setup required.
- **Local stdio server** — Run the MCP server on your machine. Requires Python and a few packages.

Agent (Claude Code, Cursor, CoWork, etc.)

```
├─ SSE transport      ── MindRouter SSE MCP Server (hosted) ── Search backend
├─ Desktop Extension  ── SSE bridge (.mcpb)    ── SSE ── MindRouter MCP Server
└─ stdio transport    ── MCP Server (local Python process) ── HTTPS ── MindRouter API
```

Available Tools

Tool	Tool Name	Description	Hosted	Local Source
------	-----------	-------------	--------	--------------

			SSE	
Web Search	<code>web_search</code>	Search the web via MindRouter (Brave Search / SearXNG)	<code>/mcp/sse</code>	agentic_ai/mcp/search/

Option 1: Hosted SSE Server (Recommended)

The simplest way to connect. MindRouter hosts the MCP server — no Python, no dependencies, no local processes. Just add a few lines to your MCP client config.

1. Get your API key

Generate an API key from the [API Keys](#) page in the MindRouter dashboard.

2. Configure your MCP client

Add this to your agent's MCP configuration (`.mcp.json`, `.cursor/mcp.json`, etc.):

```
{
  "mcpServers": {
    "mindrouter": {
      "type": "sse",
      "url": "https://mindrouter.uidaho.edu/mcp/sse",
      "headers": {
        "Authorization": "Bearer mr2_your_key_here"
      }
    }
  }
}
```

Or in Claude Code via CLI:

```
claude mcp add --transport sse \
  -H "Authorization: Bearer mr2_your_key_here" \
  mindrouter https://mindrouter.uidaho.edu/mcp/sse
```

3. Verify

In Claude Code, run `/mcp` to confirm the connection and the `web_search` tool is listed. In other clients, check your MCP server status panel.

Authentication: The hosted MCP server authenticates with your MindRouter API key via the `Authorization: Bearer` or `X-API-Key` header. Each search deducts the same token cost as the REST API.

Option 2: Claude Desktop Extension

For **Claude Desktop** (the standalone app), install the MindRouter extension for one-click setup — no terminal, no dependencies, no config files.

1. Download the extension

Download [mindrouter.mcpb](#) from the MindRouter repository.

2. Install

Double-click the `.mcpb` file, or open Claude Desktop and go to **Settings** → **Extensions** → **Advanced settings** → **Install Extension...** and select the file.

3. Configure

Enter your MindRouter API key when prompted. The server URL defaults to `https://mindrouter.uidaho.edu/mcp/sse` — change it only if you run your own MindRouter instance.

How it works: The extension runs a lightweight bridge script (bundled inside the `.mcpb`) that connects to MindRouter's hosted SSE server and translates it to the stdio transport that Claude Desktop expects. Claude Desktop provides the Node.js runtime — no local installation required.

Option 3: Local stdio Server

Run the MCP server locally if you prefer a self-hosted setup or need to work offline. The local server connects to MindRouter over HTTPS.

1. Install dependencies

```
pip install "mcp[cli]" httpx
```

2. Get your API key

Generate an API key from the [API Keys](#) page.

3. Configure in your MCP client

Claude Code CLI:

```
claude mcp add --scope user \  
  --env MINDROUTER_API_KEY=mr2_your_key_here \  
  mindrouter \  
  -- python3 /path/to/agentai/mcp/search/server.py
```

Project config (.mcp.json):

```
{  
  "mcpServers": {  
    "mindrouter": {  
      "command": "python3",  
      "args": ["/path/to/agentai/mcp/search/server.py"],  
      "env": {  
        "MINDROUTER_API_KEY": "mr2_your_key_here",  
        "MINDROUTER_BASE_URL": "https://mindrouter.uidaho.edu"  
      }  
    }  
  }  
}
```

Environment Variables (Local Server Only)

Variable	Default	Description
MINDROUTER_API_KEY	(required)	Your MindRouter API key
MINDROUTER_BASE_URL	https://mindrouter.uidaho.edu	MindRouter instance URL (change for self-hosted instances)

Connection Options Comparison

Feature	Hosted SSE	Desktop Extension	Local stdio
Best for	Claude Code, Cursor, CoWork	Claude Desktop app	Self-hosted / advanced
Setup	3-line JSON config or CLI	Double-click <code>.mcpb</code> file	Install Python + packages
Dependencies	None	None (runtime bundled)	Python 3.11+, <code>mcp[cli]</code> , <code>httpx</code>
Transport	SSE over HTTPS	SSE bridge → stdio	stdio (local process)
Technical skill	Edit JSON or run CLI	None	Python environment setup

Other MCP-Compatible Clients

Both hosted and local MCP servers work with any MCP-compatible client:

- **Cursor** — Configure in `.cursor/mcp.json`
- **CoWork** — Add in your workspace MCP configuration
- **Custom agents** — Use the [MCP Python SDK](#) client to connect programmatically

Agent Skills

MindRouter provides pre-built agent skills — markdown files containing API context, usage instructions, and example commands that AI coding agents use to interact with MindRouter services. Compatible with **Claude Code**, **ForgeCode**, **OpenCode**, **Codex**, and other agentic coding tools that support markdown-based skill or custom command definitions.

Skills are complementary to [MCP servers](#). Use skills for lightweight, dependency-free integration; use MCP servers for a standardized tool protocol with structured input/output.

Available Skills

Skill	Name	Description	Source
Web Search	<code>web-search</code>	Search the web via MindRouter's search API (Brave Search / SearXNG)	agentic_ai/skills/web-search/

Installation

Copy the skill folder into the location your agent tool expects for custom commands or skills. Each skill is a directory containing a `SKILL.md` file with YAML frontmatter and markdown instructions.

Claude Code

```
mkdir -p .claude/skills
cp -r agentic_ai/skills/web-search .claude/skills/
```

Other Agent Tools

Copy or symlink the skill directory into your tool's custom command location. If your tool doesn't have a specific skills directory, reference the `SKILL.md` file directly in your system prompt or project context — the content is standard markdown that any LLM-based agent can follow.

Configuration

Set your MindRouter API key in the environment before starting your agent:

```
export MINDROUTER_API_KEY=mr2_your_key_here
```

Optionally set a custom base URL (defaults to `https://mindrouter.uidaho.edu`):

```
export MINDROUTER_BASE_URL=https://your-instance.example.com
```

Usage

Once installed, invoke the skill via your agent’s command interface (e.g. `/web-search` in Claude Code) or ask the agent to search the web. The agent will call MindRouter’s search API, parse the results, and present titles, URLs, and snippets.

Skills vs MCP Servers

Feature	Skills	MCP (Hosted SSE)	MCP (Local stdio)
Format	Markdown + YAML frontmatter	SSE endpoint (server-hosted)	Python MCP server
Setup	Copy folder into agent’s skill directory	3-line JSON config	Install Python + packages + config
Scope	Any agent that reads markdown skill files	Any MCP-compatible client	
Dependencies	None (uses curl / shell)	None	Python, <code>mcp[cli]</code> , <code>httpx</code>

Request ID

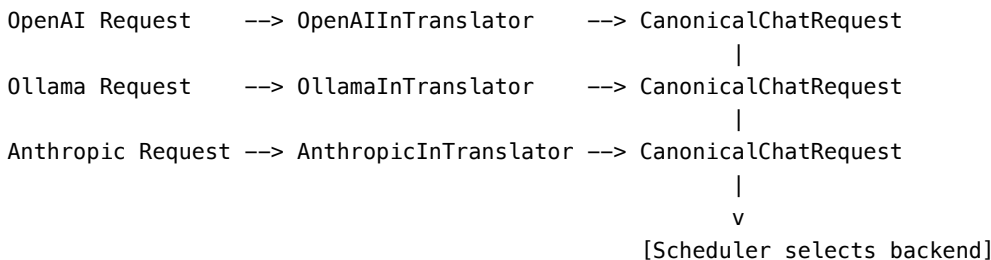
Every inference response includes a unique request ID for tracing and debugging.

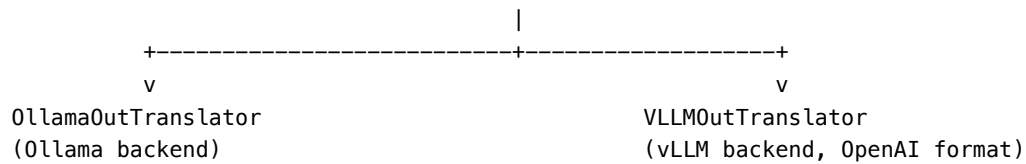
- **Auto-generated IDs** — MindRouter generates request IDs with format-specific prefixes: `chatcml-*` for chat completions, `cml-*` for text completions, `embd-*` for embeddings, etc.
- **Custom IDs** — You can provide your own request ID by setting the `X-Request-ID` header. When provided, MindRouter uses your ID instead of generating one, making it easier to correlate requests across your systems.

Translation Layer

MindRouter’s translation layer enables cross-engine routing: a request arriving in OpenAI, Ollama, or Anthropic format can be served by any Ollama or vLLM backend. All translation passes through a **canonical internal schema**.

Request Flow





Canonical Schemas

The canonical internal representation (`backend/app/core/canonical_schemas.py`) includes:

- **CanonicalChatRequest** — model, messages, temperature, top_p, max_tokens, stream, tools, tool_choice, response_format, think (bool or string), reasoning_effort, etc.
- **CanonicalMessage** — role (system/user/assistant/tool), content (text or multimodal content blocks, nullable), tool_calls, tool_call_id
- **ContentBlock** — TextContent, ImageUrlContent, or ImageBase64Content
- **CanonicalToolCall** / **CanonicalFunctionCall** — tool call with id, function name, and arguments (JSON string)
- **CanonicalToolDefinition** — tool definition with function name, description, and parameters schema
- **CanonicalEmbeddingRequest** — model, input, encoding_format, dimensions
- **CanonicalChatResponse** / **CanonicalStreamChunk** — response and streaming types (including tool call deltas)

Key Translation Mappings

Concept	OpenAI	Ollama	Anthropic	Canonical
Max tokens	<code>max_completion_tokens</code> or <code>max_tokens</code>	<code>options.num_predict</code>	<code>max_tokens</code> (required)	<code>max_tokens</code>
Stream default	<code>false</code>	<code>true</code>	<code>false</code>	—
System prompt	messages with role: system	messages with role: system	Top-level system field	<code>CanonicalMessage(role=SYSTEM)</code>
Stop sequences	<code>stop</code>	<code>options.stop</code>	<code>stop_sequences</code>	<code>stop</code>
JSON schema	<code>response_format</code>	<code>format: {schema}</code>	<code>output_config.format</code>	<code>response_format</code>
Parameters	Top-level fields	<code>options</code> dict	Top-level fields	Top-level fields
Images	<code>image_url</code> block	images array (base64)	image block with source	<code>ImageBase64Content</code> / <code>ImageUrlContent</code>
Tool definitions	<code>tools</code>	<code>tools</code>	<code>tools</code> (with <code>input_schema</code>)	<code>tools</code> (<code>CanonicalToolDefinition</code>)
Tool choice	<code>tool_choice</code>	—	<code>tool_choice</code> (auto/any/tool)	<code>tool_choice</code>
Tool calls	<code>tool_calls</code> (JSON string args)	<code>tool_calls</code> (dict args)	<code>tool_use</code> content blocks	<code>CanonicalToolCall</code> (JSON string args)
Tool results	role: "tool" + <code>tool_call_id</code>	—	<code>tool_result</code> content blocks	<code>CanonicalMessage(role=TOOL, tool_call_id)</code>
Thinking mode	<code>think</code> , <code>thinking.type</code> , <code>chat_template_kwargs</code> , <code>reasoning_effort</code>	<code>think</code> (bool or "low"/"medium"/"high")	<code>thinking.type</code> (enabled/disabled)	<code>think</code> (bool or string)

Concept	OpenAI	Ollama	Anthropic	Canonical
User ID	<code>user</code>	—	<code>metadata.user_id</code>	<code>user</code>
Stream format	SSE (<code>data: {...}</code>)	NDJSON	SSE (Anthropic events)	<code>CanonicalStreamChunk</code>

Translators

Translator	Direction	Purpose
<code>OpenAIInTranslator</code>	API → Canonical	Translates incoming OpenAI-format requests
<code>OllamaInTranslator</code>	API → Canonical	Translates incoming Ollama-format requests
<code>AnthropicInTranslator</code>	API → Canonical	Translates incoming Anthropic Messages API requests; also formats responses and SSE stream events back to Anthropic format
<code>OllamaOutTranslator</code>	Canonical → Backend	Translates outgoing requests to Ollama backends
<code>VLLMOutTranslator</code>	Canonical → Backend	Translates outgoing requests to vLLM backends

All translators use static methods — no instantiation needed.

Model-Specific Behaviors

- **Qwen3-32B on vLLM** — This model embeds `<think>...</think>` tags directly in the content field instead of using the `reasoning_content` field. MindRouter automatically extracts these tags and moves the reasoning text to the canonical `reasoning` field for both streaming and non-streaming responses.
- **Qwen3.5 on vLLM with thinking disabled** — When thinking is disabled but the model still returns reasoning content with an empty content field, MindRouter promotes the reasoning content to the content field so the response is not blank.

Telemetry & Monitoring

GPU Sidecar Agent

Each GPU node runs a lightweight FastAPI sidecar agent (`sidecar/gpu_agent.py`) that exposes per-GPU hardware metrics:

Collected metrics per GPU:

- Utilization (GPU % and memory %)
- Memory (used/free/total GB)
- Temperature (GPU and memory)
- Power draw and limit (watts)
- Fan speed, SM/memory clocks
- Running processes (PID + memory)
- Device identity (name, UUID, compute capability)
- Driver and CUDA versions

Authentication: Requires `SIDECAR_SECRET_KEY` env var. All requests must include `X-Sidecar-Key` header (constant-time

comparison).

Prerequisites: Each GPU node must have NVIDIA drivers and the [NVIDIA Container Toolkit](#) installed so Docker can access GPUs via `--gpus all`:

```
# RHEL/Rocky Linux
curl -s -L https://nvidia.github.io/libnvidia-container/stable/rpm/nvidia-container-toolkit.repo \
  | sudo tee /etc/yum.repos.d/nvidia-container-toolkit.repo
sudo dnf install -y nvidia-container-toolkit

# Debian/Ubuntu
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
  | sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list \
  | sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https' \
  | sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
sudo apt-get update && sudo apt-get install -y nvidia-container-toolkit

# Configure and restart Docker
sudo nvidia-ctk runtime configure --driver=docker
sudo systemctl restart docker

# Verify GPU access
docker run --rm --gpus all nvidia/cuda:12.2.0-base-ubuntu22.04 nvidia-smi
```

Deployment options:

1. **Docker Compose** — `docker compose --profile gpu up gpu-sidecar`
2. **Standalone Docker** — Build from `sidecar/Dockerfile.sidecar`, run with `--gpus all`
3. **Direct Python** — `pip install fastapi uvicorn nvidia-ml-py && python sidecar/gpu_agent.py`

Health Polling

The Backend Registry runs an adaptive polling loop:

- **Startup fast polls:** On container start, two immediate full poll cycles run (with a 5-second gap) so backends and nodes are marked healthy within seconds of a restart instead of waiting for the normal 30-second cycle.
- **Normal interval:** 30 seconds (configurable via `BACKEND_POLL_INTERVAL`)
- **Fast interval:** 10 seconds after a backend becomes unhealthy (configurable)
- **Fast duration:** 120 seconds before returning to normal polling

Each poll cycle has two phases:

1. Poll sidecar agents (one per physical node) for GPU snapshots
2. Poll each backend adapter for health, models, and engine-specific telemetry

Health Alerts

The admin dashboard (`/admin`) displays a prominent warning banner when any backend is unhealthy/unknown or any node is offline/unknown. The alert includes counts and names of affected items with direct links to the backends or nodes management pages. Intentionally **disabled** backends are excluded from the alert — only unexpected health issues are flagged.

Circuit Breaker

Per-backend circuit breaker protects against cascading failures:

- **Threshold:** 3 consecutive failures before opening (configurable via `BACKEND_CIRCUIT_BREAKER_THRESHOLD`)
- **Recovery:** 30 seconds before allowing a probe request (`BACKEND_CIRCUIT_BREAKER_RECOVERY_SECONDS`)
- **States:** Closed (healthy) → Open (failing) → Half-Open (probe) → Closed (recovered)

Latency Tracking

Exponential Moving Average (EMA) tracks per-backend latency:

- **Alpha:** 0.3 (30% current observation, 70% history)
- **Metrics:** Total latency EMA and TTFT (time-to-first-token) EMA
- **Throughput score:** $1.0 / (1.0 + \text{latency_ms} / 5000.0)$ — used in backend scoring
- **Persistence:** EMAs are periodically saved to the database for recovery after restart

Prometheus Metrics

Scrape `/metrics` for Prometheus-compatible metrics. See the [Health & Metrics Endpoints](#) section for the full list.

Telemetry API

Admin users can access detailed telemetry via the API:

- **Cluster overview** — All nodes, backends, GPUs with current metrics
- **Historical data** — Time-series with configurable resolution (1m, 5m, 15m, 1h, 6h, 1d)
- **Per-GPU history** — Individual GPU device telemetry over time
- **Export** — Download telemetry data as JSON or CSV

Redis (Optional)

Redis is an optional dependency configured via `REDIS_URL`. When available, it serves three roles:

- **Inflight streaming token counting** — Atomically tracks tokens currently being streamed across all workers, enabling accurate real-time throughput metrics.
- **Per-user quota token caching** — Caches per-user token counters (`quota:tokens:{user_id}`) for fast atomic increments without hitting the database on every request.
- **RPM rate limiting** — Cluster-wide requests-per-minute enforcement via atomic INCR + EXPIRE counters (`rpm:user:{user_id}`) with a 60-second sliding window. Shared across all workers.
- **Graceful degradation** — All features that depend on Redis continue to work without it. Token quota enforcement falls back to database queries, RPM enforcement fails open (requests are allowed), and inflight token counts default to zero. No functionality is lost; only caching and rate limiting benefits are reduced.

Data Retention

MindRouter includes a two-tier data retention system that automatically archives and purges old data to keep the primary database lean while preserving historical records in a separate archive database.

Architecture

The retention system manages three data categories:

Category	Tables	Default Tier 1	Default Tier 2
Requests	<code>requests</code> , <code>responses</code> , <code>scheduler_decisions</code> , <code>artifacts</code>	90 days	730 days (2 years)

Category	Tables	Default Tier 1	Default Tier 2
Chat	chat_conversations, chat_messages, chat_attachments	90 days	730 days (2 years)
Telemetry	backend_telemetry, gpu_device_telemetry	30 days	Not archived (deleted only)

- **Tier 1 (App DB)** — Data lives in the primary database for the configured retention period. When it expires, it is archived to Tier 2 (if configured) then deleted from the app DB.
- **Tier 2 (Archive DB)** — A separate MariaDB instance that stores full copies of archived data. After the Tier 2 retention period expires, archived data is permanently purged.

Archive Database

The archive database is a second MariaDB container (`mariadb-archive`) defined in `docker-compose.yml`. It runs alongside the primary database and stores archived records in FK-free tables that mirror the app schema.

The archive DB is **optional**. If `ARCHIVE_DATABASE_URL` is not set, the retention system still works — expired data is simply deleted without archival.

Configuration

Add the following environment variable to enable archival:

```
ARCHIVE_DATABASE_URL=mysql+pymysql://mindrouter_archive:<password>@mariadb-archive:3306/mindrouter_arc
```

The Docker Compose files include a `mariadb-archive` service that is started automatically. In production, configure the archive password variables in your `.env.prod` file:

```
MYSQL_ARCHIVE_ROOT_PASSWORD=<secure-root-password>
MYSQL_ARCHIVE_PASSWORD=<secure-password>
ARCHIVE_DATABASE_URL=mysql+pymysql://mindrouter_archive:<secure-password>@mariadb-archive:3306/mindrou
```

Archive tables are created automatically at startup via `create_all(checkfirst=True)` — no Alembic migration is needed for the archive database.

Retention Policies

All retention policies are stored in the `AppConfig` table and are editable at runtime from the admin UI at `/admin/retention`. No restart is required when changing policies.

AppConfig Key	Default	Description
<code>retention.requests.tier1_days</code>	90	Days before request data is archived
<code>retention.requests.tier2_days</code>	730	Days archived requests are kept before purge
<code>retention.chat.tier1_days</code>	90	Days before chat data is archived
<code>retention.chat.tier2_days</code>	730	Days archived chat data is kept before purge
<code>retention.telemetry.tier1_days</code>	30	Days before telemetry is purged (no archival)
<code>retention.telemetry.tier2_days</code>	0	0 = telemetry is not archived
<code>retention.cleanup_interval</code>	3600	Seconds between retention cycle runs

AppConfig Key	Default	Description
<code>retention.batch_size</code>	500	Rows per archive+delete batch

Setting a Tier 1 value to 0 disables retention for that category (data is kept indefinitely).

Admin UI

The **Data Retention** page in the admin dashboard (`/admin/retention`) provides three tabs:

- **Policies** — Edit all retention periods and advanced settings (cleanup interval, batch size). Shows current row counts in the app DB for each category.
- **Archive Stats** — Per-table row counts and disk usage in the archive database. Only available when the archive DB is configured.
- **Browse Archive** — Paginated view of archived records with filters for model and user. Supports browsing both archived requests and chat conversations.

A **Run Retention Now** button triggers an immediate retention cycle without waiting for the next scheduled run.

Background Loop

A background task (`_retention_loop`) runs continuously and executes a full retention cycle at the configured interval. Each cycle:

1. Archives and deletes expired requests from the app DB (Tier 1)
2. Archives and deletes expired chat conversations from the app DB (Tier 1)
3. Deletes expired telemetry data from the app DB (no archival)
4. Purges expired records from the archive DB (Tier 2)
5. Cleans up orphan chat attachments

All operations are batched and use FK-order deletion to maintain referential integrity. Archive inserts use `INSERT IGNORE` for idempotent re-runs.

Storage

The archive database volume can be mounted to a separate, larger disk. In production, `docker-compose.prod.yml` uses a host bind mount (e.g., `/archivedb:/var/lib/mysql`) so the archive data can reside on a different filesystem than the primary database.

Chat System

MindRouter includes a built-in chat interface at `/chat` with full conversation management.

Conversations

- Each user has their own conversation history
- Conversations store: title, selected model, creation/update timestamps
- Users can rename, switch models, or delete conversations
- Up to 50 conversations shown in the sidebar (most recent first)

Messages

- Messages include role (user/assistant/system) and content
- Assistant messages are streamed in real-time
- Messages can be edited or deleted after creation
- Attachments are linked to individual messages

File Upload

Supported file types and processing:

Category	Extensions	Processing
Images	<code>.jpg</code> , <code>.jpeg</code> , <code>.png</code> , <code>.gif</code> , <code>.webp</code>	Resized to max 1536px, compressed JPEG q85, thumbnail generated
Documents	<code>.pdf</code>	Text extracted from all pages, first-page thumbnail generated
Documents	<code>.docx</code>	Text extracted from all paragraphs
Spreadsheets	<code>.xlsx</code>	All sheets read, formatted as tab-separated text
Text files	<code>.txt</code> , <code>.md</code> , <code>.csv</code> , <code>.json</code> , <code>.html</code> , <code>.htm</code> , <code>.log</code>	Read as-is

Limits:

- Max upload size: 10 MB (configurable via `CHAT_UPLOAD_MAX_SIZE_MB`)
- Artifact storage path: `/data/artifacts` (configurable via `ARTIFACT_STORAGE_PATH`)
- Artifact max size: 50 MB (configurable via `ARTIFACT_MAX_SIZE_MB`)
- Artifact retention: 365 days

Storage layout:

```
/artifacts/YYYY/MM/DD/<sha256_prefix>/<full_sha256>_<uuid>.<ext>
```

Multimodal Model Support

- Models with multimodal capability are automatically detected by name patterns (e.g., `llava`, `-vl-`, `vision`)
- When images are sent to a multimodal model, they are included as base64-encoded content blocks
- When images are sent to a non-multimodal model, they are replaced with a placeholder: `[Image omitted -- model does not support multimodal input: filename]`
- A warning modal is shown in the chat UI when uploading images to a non-multimodal model
- Admins can override multimodal detection per model via the Models admin page

Streaming

Chat responses are streamed in real-time:

- Backend streaming uses NDJSON (Ollama) or SSE (vLLM/OpenAI)
- The chat UI renders tokens as they arrive
- TTFT (time-to-first-token) is tracked for latency monitoring
- If a backend request times out or the client disconnects, MindRouter closes the backend connection, signalling the backend to abort generation and free the slot

Conversation Retention

Conversations older than `CONVERSATION_RETENTION_DAYS` (default 730 days / 2 years) are automatically purged by a

background task that runs every `CONVERSATION_CLEANUP_INTERVAL` seconds (default 86400 / once per day). Associated messages and attachments are deleted along with the conversation.

Voice (TTS / STT)

MindRouter provides OpenAI-compatible **Text-to-Speech** (TTS) and **Speech-to-Text** (STT) endpoints that proxy to self-hosted GPU services. Audio is processed entirely on institutional infrastructure — no data leaves the cluster.

Overview

- **TTS** — Powered by [Kokoro](#) (or any service exposing an OpenAI-compatible `/v1/audio/speech` endpoint)
- **STT** — Powered by [faster-whisper](#) / [Speeches](#) (or any service exposing an OpenAI-compatible `/v1/audio/transcriptions` endpoint)
- Both endpoints require API key authentication (same keys used for chat/completions)
- Each request is logged with its own modality (`tts` or `stt`) and a configurable token cost is deducted from the user's quota
- The built-in Chat UI integrates both features: **Read Aloud** buttons on assistant messages (TTS) and a **microphone button** for voice input (STT)

API Endpoints

Text-to-Speech — `POST /v1/audio/speech`

Converts text to audio. The response is a streaming audio file.

Request body (JSON):

Field	Type	Default	Description
<code>model</code>	string	"kokoro"	TTS model name (must match the upstream service)
<code>input</code>	string	<i>required</i>	The text to synthesize
<code>voice</code>	string	"af_heart"	Voice ID (see available voices below)
<code>response_format</code>	string	"mp3"	Audio format: <code>mp3</code> , <code>wav</code> , <code>opus</code> , or <code>flac</code>
<code>speed</code>	float	1.0	Playback speed (0.25 – 4.0)

Example (curl):

```
curl -X POST https://mindrouter.uidaho.edu/v1/audio/speech \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "kokoro",
    "input": "Hello from MindRouter!",
    "voice": "af_heart",
    "response_format": "mp3",
    "speed": 1.0
  }' \
  -o output.mp3
```

Example (Python — OpenAI SDK):

```

from openai import OpenAI

client = OpenAI(
    base_url="https://mindrouter.uidaho.edu/v1",
    api_key="YOUR_API_KEY",
)

response = client.audio.speech.create(
    model="kokoro",
    voice="af_heart",
    input="Hello from MindRouter!",
    response_format="mp3",
    speed=1.0,
)

response.stream_to_file("output.mp3")

```

Response: Streaming audio bytes with **Content-Type** matching the requested format (e.g., **audio/mpeg** for mp3).

Speech-to-Text — **POST** /v1/audio/transcriptions

Transcribes an audio file to text.

Request (multipart form):

Field	Type	Default	Description
file	file upload	<i>required</i>	Audio file (mp3, wav, webm, m4a, ogg, flac, etc.)
model	string	<i>server default</i>	STT model name (omit to use the admin-configured default)
language	string	<i>auto-detect</i>	ISO 639-1 language code (e.g., en , es , fr)
response_format	string	"json"	Output format (see table below)

Response formats:

Format	Content-Type	Description
json	application/json	Simple JSON: {"text": "..."}
verbose_json	application/json	JSON with word-level timestamps, segments, language, and duration
text	text/plain	Plain text transcription only
srt	text/plain	SubRip subtitle format with timestamps
vtt	text/vtt	WebVTT subtitle format with timestamps

Example (curl):

```

curl -X POST https://mindrouter.uidaho.edu/v1/audio/transcriptions \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -F "file=@recording.mp3" \
  -F "language=en" \
  -F "response_format=verbose_json"

```

Example (Python — OpenAI SDK):

```

from openai import OpenAI

```

```

client = OpenAI(
    base_url="https://mindrouter.uidaho.edu/v1",
    api_key="YOUR_API_KEY",
)

with open("recording.mp3", "rb") as audio_file:
    transcript = client.audio.transcriptions.create(
        model="default",
        file=audio_file,
        language="en",
        response_format="verbose_json",
    )
print(transcript.text)

```

Response examples:

json:

```
{"text": "Hello from MindRouter!"}
```

verbose_json:

```
{
  "task": "transcribe",
  "language": "english",
  "duration": 3.42,
  "text": "Hello from MindRouter!",
  "segments": [
    {
      "id": 0,
      "start": 0.0,
      "end": 3.42,
      "text": "Hello from MindRouter!",
      ...
    }
  ]
}
```

srt:

```

1
00:00:00,000 --> 00:00:03,420
Hello from MindRouter!

```

vtt:

```

WEBVTT

00:00:00.000 --> 00:00:03.420
Hello from MindRouter!

```

List Voices — GET /api/tts-voices

Returns the available TTS voices. When the upstream Kokoro service is reachable, the list is fetched dynamically; otherwise it falls back to the admin-configured voice list.

Parameter	Type	Default	Description
<code>allowed_only</code>	bool	false	If true, only return voices in the admin-configured allowed list

Response (JSON):

```
{
  "voices": ["af_bella", "af_heart", "am_adam", "am_michael"],
  "source": "upstream",
  "default_voice": "af_bella"
}
```

Voices

Kokoro ships with multiple voices out of the box. The admin can restrict which voices are available to users via the [Voice Config](#) admin page. Some commonly used voices:

Voice ID	Gender	Description
af_heart	Female	Default voice — warm and natural
af_bella	Female	Clear and articulate
am_adam	Male	Neutral male voice
am_michael	Male	Deeper male voice

The full list of available voices depends on the Kokoro version deployed. Use the `GET /api/tts-voices` endpoint to discover all available voices dynamically.

Chat UI Integration

When TTS and/or STT are enabled by an admin, the built-in chat interface at `/chat` gains voice features:

Read Aloud (TTS)

- A **Read aloud** button appears below each assistant message
- Clicking it streams the response text through the TTS service, sentence by sentence, playing audio sequentially
- Markdown formatting (code blocks, links, headers, bold/italic) is automatically stripped before synthesis so only natural text is spoken
- During streaming responses, TTS begins as soon as the first complete sentence arrives — it does not wait for the full response
- Clicking the button again (or starting a new Read Aloud) stops the current playback
- An animated border around the input area indicates when TTS is playing (cyan/blue/purple glow)

Voice Input (STT)

- A **microphone button** appears in the input bar when STT is enabled
- Clicking it requests browser microphone permission, then starts recording
- The button turns red during recording and an animated border (magenta/rose/amber) indicates active capture
- Clicking the button again stops recording, sends the audio to the STT service, and inserts the transcribed text into the message input
- Audio format is negotiated automatically (prefers WebM/Opus, falls back to WebM or MP4)
- The STT service timeout is 120 seconds to accommodate longer recordings or cold-start model loading

User Preferences

When TTS is enabled, users can customize their voice experience from their **User Dashboard** (`/dashboard`):

- **TTS Voice** — Choose from the admin-allowed voice list, or leave as system default
- **TTS Speed** — Slider from 0.5x to 2.0x playback speed, with a reset-to-default button

Per-user preferences are stored in the `app_config` table as `user.{id}.tts_voice` and `user.{id}.tts_speed`. They take priority over the system-wide defaults but can be overridden per-request via the API.

Admin Configuration

Admins manage voice settings from two locations:

Chat Config (`/admin/chat-config`)

High-level on/off toggles and system-wide defaults:

Setting	Config Key	Default	Description
TTS Enabled	<code>voice.tts_enabled</code>	<code>false</code>	Master switch for TTS in the chat UI and API

Setting	Config Key	Default	Description
TTS Provider	<code>voice.tts_provider</code>	"kokoro"	Model name sent to the upstream TTS service
TTS Voice	<code>voice.tts_voice</code>	"af_heart"	System-wide default voice (lowest priority fallback)
TTS Speed	<code>voice.tts_speed</code>	1.0	System-wide default speed
STT Enabled	<code>voice.stt_enabled</code>	false	Master switch for STT in the chat UI and API

Voice Config (/admin/voice-config)

Backend connection details and quota settings:

TTS Backend:

Setting	Config Key	Default	Description
Service URL	<code>voice.tts_url</code>	none	Base URL of the TTS service (e.g., <code>http://kokoro:8880</code>)
API Key	<code>voice.tts_api_key</code>	none	Bearer token for the upstream TTS service (if required)
Available Voices	<code>voice_api.tts_voices</code>	af_heart, af_bella, am_adam, am_michael	Newline-separated list of voice IDs users can select
Default Voice	<code>voice_api.default_voice</code>	"af_heart"	Voice assigned to new users who haven't chosen one

STT Backend:

Setting	Config Key	Default	Description
Service URL	<code>voice.stt_url</code>	none	Base URL of the STT service (e.g., <code>http://whisper:8000</code>)
API Key	<code>voice.stt_api_key</code>	none	Bearer token for the upstream STT service (if required)
Default Model	<code>voice.stt_model</code>	"deepdml/faster-whisper-large-v3-turbo-ct2"	Whisper model passed to the upstream service

Quota:

Setting	Config Key	Default	Description
TTS tokens per request	<code>voice_api.tts_quota_tokens</code>	100	Fixed token cost deducted from user quota for each TTS call
STT tokens per request	<code>voice_api.stt_quota_tokens</code>	200	Fixed token cost deducted from user quota for each STT call

Voice requests use a **fixed token cost** model rather than counting actual tokens, since audio processing does not map cleanly to token counts. Adjust these values to control how heavily voice usage counts against a user's quota budget.

Voice Priority Resolution

When a TTS request is made (via the API or the Chat UI), the voice is resolved in the following order:

1. **Per-request** — `voice` field in the API request body

2. **Per-user preference** — `user.{id}.tts_voice` (set from the user dashboard)
3. **Admin default voice** — `voice_api.default_voice` (set in Voice Config)
4. **System fallback** — `voice.tts_voice` (set in Chat Config)
5. **Hardcoded fallback** — `"af_heart"`

Speed follows a similar chain: per-user preference → system default → `1.0`.

Error Handling

HTTP Status	Condition
400	Empty or blank input text (TTS)
404	TTS or STT is not enabled by the admin
429	User's token quota exceeded or RPM rate limit hit
500	TTS/STT service URL not configured
502	Upstream TTS/STT service returned an error, timed out, or is unreachable

Failed requests are recorded with the error message but do not deduct tokens from the user's quota.

Blog / CMS

MindRouter includes a built-in blog system with public viewing and admin content management. Blog posts are written in Markdown and rendered to HTML with syntax highlighting, tables, fenced code blocks, and table of contents support.

Public Blog

- **Blog listing** at `/blog` — displays all published posts, most recent first
- **Individual posts** at `/blog/{slug}` — renders the post's Markdown content as styled HTML
- Posts are accessible without authentication

Admin Blog Management

Admin users can manage blog posts at `/admin/blog`. The admin interface provides a full CMS workflow:

Action	Route	Description
List all posts	GET <code>/admin/blog</code>	View all posts (published, draft, and soft-deleted) with status indicators
New post	GET <code>/admin/blog/new</code>	Form to create a new blog post
Create post	POST <code>/admin/blog/new</code>	Submit new post with title, slug, Markdown content, excerpt, and publish toggle
Edit post	GET <code>/admin/blog/{id}/edit</code>	Edit form for an existing post
Update post	POST <code>/admin/blog/{id}/edit</code>	Save changes to a post
Toggle publish	POST <code>/admin/blog/{id}/publish</code>	Publish a draft or unpublish a live post
Delete post	POST <code>/admin/blog/{id}/delete</code>	Soft-delete a post (not permanently removed from the database)

Post Fields

- **Title** — Display title of the post
- **Slug** — URL-safe identifier (auto-generated from title or manually set). Used in the public URL as `/blog/{slug}`
- **Content** — Full post body written in Markdown. Supports fenced code blocks with syntax highlighting, tables, and table of contents
- **Excerpt** — Optional short summary shown in the blog listing
- **Published** — Toggle for publish state. The `published_at` timestamp is set automatically when a post is first published
- **Author** — Automatically set to the admin user who creates the post

Soft Delete

Deleting a post is a soft delete — the post is marked as deleted but remains in the database. Soft-deleted posts are hidden from the public blog listing but still visible in the admin list.

Blog Email Notifications

When publishing a blog post, admins can optionally send an email notification to all users who have opted in to blog email notifications. The email contains the full post rendered as HTML with a "Read on the Web" link. Users can manage their email preference from their dashboard settings.

Email System

MindRouter includes an email system for admin-to-user communication. Emails are sent asynchronously in the background and support personalization, HTML formatting, and recipient targeting.

SMTP Configuration

Email requires SMTP credentials configured via the **Admin > Settings** page. The following AppConfig keys control SMTP:

Key	Description
<code>email.smtp_host</code>	SMTP server hostname
<code>email.smtp_port</code>	SMTP server port (default: 587)
<code>email.smtp_username</code>	SMTP authentication username
<code>email.smtp_password</code>	SMTP authentication password
<code>email.use_tls</code>	Enable STARTTLS (default: true)
<code>email.default_sender</code>	Default "From" address for emails
<code>email.blog_sender</code>	"From" address for blog notification emails (falls back to <code>default_sender</code>)
<code>email.test_address</code>	Recipient address for test emails

Composing Emails

Admin users can compose and send emails at `/admin/email`. The interface supports three recipient modes:

- **All Users** — Send to every registered user
- **By Group** — Send to all members of a selected group
- **Select Users** — Search and pick individual recipients

Template Variables

The subject and body support personalization variables that are replaced per-recipient:

Variable	Replaced With
	User's first name (from full_name)
	User's last name (from full_name)
	User's username
	User's email address

Email Delivery

Emails are sent asynchronously as a background task. A small throttle delay (50ms) is inserted between recipients to avoid overwhelming the SMTP server. Delivery progress is tracked in an email log visible at the bottom of the `/admin/email` page, showing status (sending, completed, failed), success/failure counts, and any error messages.

Test Emails

Before sending to real users, admins can send a test email to the configured test address to verify SMTP connectivity and email rendering.

AppConfig System

MindRouter stores runtime configuration in a key-value `AppConfig` database table. Values are JSON-encoded and can be read or written at any time without restarting the application. This powers several features that need runtime-adjustable settings.

API

Configuration is accessed via two async CRUD functions in `backend/app/db/crud.py`:

- `get_config_json(db, key, default)` — Read a config value, JSON-decoded. Returns `default` if the key does not exist.
- `set_config(db, key, value, description=None)` — Upsert a config value (JSON-encoded). Creates the key if it does not exist, updates it otherwise.

Known Configuration Keys

Key	Type	Default	Description	Managed Via
<code>app.timezone</code>	string	<code>None</code>	Display timezone for the dashboard (e.g. "America/Los_Angeles")	Site Settings
<code>ollama.enforce_num_ctx</code>	bool	<code>true</code>	Whether MindRouter overrides user-supplied <code>num_ctx</code> values	Site Settings

Key	Type	Default	Description	Managed Via
<code>chat.core_models</code>	list	<code>[]</code>	Subset of models shown in the chat interface model selector. Empty means show all.	Chat Configuration
<code>chat.default_model</code>	string	<code>null</code>	Pre-selected model in the chat interface	Chat Configuration
<code>chat.system_prompt</code>	string	<code>null</code>	Global system prompt prepended to all chat conversations. Blank removes the override.	Chat Configuration
<code>chat.max_tokens</code>	int	<code>16384</code>	Default max tokens for chat requests (range: 256–131072)	Chat Configuration
<code>chat.temperature</code>	float	<code>null</code>	Default temperature for chat requests (range: 0.0–2.0). Null means use model default.	Chat Configuration
<code>chat.think</code>	bool/ string	<code>null</code>	Default thinking mode for chat: <code>true</code> , <code>false</code> , <code>"low"</code> , <code>"medium"</code> , <code>"high"</code> , or <code>null</code> (no default)	Chat Configuration
<code>catalog.auto_enrich</code>	bool	<code>false</code>	Enable automatic model description enrichment	Site Settings
<code>catalog.enrich_model</code>	string	<code>""</code>	Model name used for LLM enrichment calls	Site Settings
<code>catalog.enrich_api_key</code>	string	<code>""</code>	API key for internal MindRouter enrichment calls	Site Settings
<code>catalog.brave_api_key</code>	string	<code>""</code>	Brave Search API key for enrichment (overrides env var)	Site Settings
<code>search.enabled</code>	bool	<code>true</code>	Enable/disable the <code>/v1/search</code> API	Search
<code>search.provider</code>	string	<code>"brave"</code>	Active search provider: <code>brave</code> or <code>searxng</code>	Search
<code>search.max_results</code>	int	<code>10</code>	Default max results per search (clients can override up to 50)	Search
<code>search.quota_tokens_per_request</code>	int	<code>50</code>	Fixed token cost deducted per search request	Search
<code>search.brave.api_key</code>	string	<code>""</code>	Brave Search API key (falls back to <code>BRAVE_SEARCH_API_KEY</code> env var)	Search

Key	Type	Default	Description	Managed Via
<code>search.brave.endpoint</code>	string	<code>https://api.search.brave.com/...</code>	Brave Search API endpoint URL	Search
<code>search.searxng.endpoint</code>	string	<code>""</code>	SearXNG instance URL (must have JSON format enabled)	Search
<code>email.smtp_host</code>	string	<code>""</code>	SMTP server hostname	Site Settings
<code>email.smtp_port</code>	int	<code>587</code>	SMTP server port	Site Settings
<code>email.smtp_username</code>	string	<code>""</code>	SMTP authentication username	Site Settings
<code>email.smtp_password</code>	string	<code>""</code>	SMTP authentication password	Site Settings
<code>email.use_tls</code>	bool	<code>true</code>	Enable STARTTLS for SMTP	Site Settings
<code>email.default_sender</code>	string	<code>""</code>	Default "From" address for emails	Site Settings
<code>email.blog_sender</code>	string	<code>""</code>	"From" address for blog notification emails	Site Settings
<code>email.test_address</code>	string	<code>""</code>	Recipient address for test emails	Site Settings

All AppConfig values take effect immediately upon save — no application restart is required. These settings are managed via the **Admin Dashboard** under Site Settings and Chat Configuration.

Configuration Reference

All settings are loaded from environment variables or `.env` / `.env.prod` files. Variable names are case-insensitive.

Application

Variable	Type	Default	Description
<code>APP_NAME</code>	str	<code>MindRouter</code>	Application name
<code>APP_VERSION</code>	str	<i>(from pyproject.toml)</i>	Application version (read dynamically at startup)
<code>DEBUG</code>	bool	<code>false</code>	Enable debug mode
<code>RELOAD</code>	bool	<code>false</code>	Auto-reload on code changes (development)

Database

Variable	Type	Default	Description
<code>DATABASE_URL</code>	str	<code>mysql+pymysql://...</code>	MariaDB/MySQL connection string
<code>DATABASE_POOL_SIZE</code>	int	<code>30</code>	Connection pool size
<code>DATABASE_MAX_OVERFLOW</code>	int	<code>20</code>	Max overflow connections beyond pool
<code>DATABASE_ECHO</code>	bool	<code>false</code>	Log SQL queries

Variable	Type	Default	Description
ARCHIVE_DATABASE_URL	str	None	Archive MariaDB connection string (optional — enables tiered archival)

Cache

Variable	Type	Default	Description
REDIS_URL	str	None	Redis connection string (optional)

Security

Variable	Type	Default	Description
SECRET_KEY	str	dev-secret-key-...	JWT/session signing key (change in production)
JWT_ALGORITHM	str	HS256	JWT signing algorithm
JWT_EXPIRATION_HOURS	int	24	JWT token lifetime
SESSION_COOKIE_NAME	str	mindrouter_session	Session cookie name
SESSION_COOKIE_SECURE	bool	false	HTTPS-only cookies
SESSION_COOKIE_HTTPONLY	bool	true	JavaScript-inaccessible cookies
SESSION_COOKIE_SAMESITE	str	lax	SameSite cookie policy
API_KEY_HASH_ALGORITHM	str	argon2	API key hashing algorithm

Azure AD SSO (Optional)

Variable	Type	Default	Description
AZURE_AD_CLIENT_ID	str	None	Azure AD application (client) ID
AZURE_AD_CLIENT_SECRET	str	None	Azure AD client secret
AZURE_AD_TENANT_ID	str	None	Azure AD tenant ID
AZURE_AD_REDIRECT_URI	str	https://<host>/login/azure/authorized	OAuth2 redirect URI
AZURE_AD_DEFAULT_GROUP	str	other	Default group for new Azure AD users

When `AZURE_AD_CLIENT_ID` and `AZURE_AD_TENANT_ID` are set, a "Sign in with Microsoft" button appears on the login page. Users are JIT-provisioned on first login — their `jobTitle` from Microsoft Graph determines group assignment (student/faculty/staff/other). Pre-existing accounts are linked by email.

Azure AD Group Mapping

Group assignment uses **substring matching** on the user's `jobTitle` field from Microsoft Graph:

- `jobTitle` contains "**student**" → `students` group
- `jobTitle` contains "**faculty**" or "**professor**" → `faculty` group
- `jobTitle` contains "**staff**" → `staff` group
- No match → falls back to `AZURE_AD_DEFAULT_GROUP` (default: `other`)

Web Search (Optional)

Variable	Type	Default	Description
----------	------	---------	-------------

Variable	Type	Default	Description
BRAVE_SEARCH_API_KEY	str	None	Brave Search API key — used as fallback for both the chat web search toggle and the <code>/v1/search</code> API if no key is configured in Admin → Search
BRAVE_SEARCH_MAX_RESULTS	int	5	Maximum search results to inject as context (chat toggle only)

When configured, a search toggle appears in the chat input area. Enabling it queries the Brave Search API with the user's message and injects results into the system prompt as additional context.

The standalone `/v1/search` API ([see Web Search](#)) is configured separately via **Admin → Search** and supports multiple providers (Brave, SearXNG). The `BRAVE_SEARCH_API_KEY` env var serves as a fallback if no key is set in the admin UI.

Conversation Retention

Variable	Type	Default	Description
CONVERSATION_RETENTION_DAYS	int	730	Conversation retention period (2 years)
CONVERSATION_CLEANUP_INTERVAL	int	86400	Cleanup interval in seconds (24 hours)

Artifact Storage

Variable	Type	Default	Description
ARTIFACT_STORAGE_PATH	str	<code>/data/artifacts</code>	File storage directory
ARTIFACT_MAX_SIZE_MB	int	50	Max artifact file size
ARTIFACT_RETENTION_DAYS	int	365	Artifact retention period

Quotas

Quota defaults are now managed per-group in the database via `/admin/groups`. The environment variables below are **deprecated** (used only for initial migration seeding) and will be removed in a future release.

Variable	Type	Default	Description
DEFAULT_TOKEN_BUDGET_STUDENT	int	100000	<i>Deprecated</i> — use group defaults
DEFAULT_TOKEN_BUDGET_STAFF	int	500000	<i>Deprecated</i> — use group defaults
DEFAULT_TOKEN_BUDGET_FACULTY	int	1000000	<i>Deprecated</i> — use group defaults
DEFAULT_TOKEN_BUDGET_ADMIN	int	10000000	<i>Deprecated</i> — use group defaults
DEFAULT_RPM_STUDENT	int	30	<i>Deprecated</i> — use group defaults
DEFAULT_RPM_STAFF	int	60	<i>Deprecated</i> — use group defaults
DEFAULT_RPM_FACULTY	int	120	<i>Deprecated</i> — use group defaults
DEFAULT_RPM_ADMIN	int	1000	<i>Deprecated</i> — use group defaults

Scheduler

Scheduler weights are now managed per-group in the database. The per-role weight variables below are **deprecated** and will be removed in a future release.

Variable	Type	Default	Description
SCHEDULER_WEIGHT_STUDENT	int	1	<i>Deprecated</i> — use group scheduler_weight

Variable	Type	Default	Description
SCHEDULER_WEIGHT_STAFF	int	2	<i>Deprecated</i> — use group scheduler_weight
SCHEDULER_WEIGHT_FACULTY	int	3	<i>Deprecated</i> — use group scheduler_weight
SCHEDULER_WEIGHT_ADMIN	int	10	<i>Deprecated</i> — use group scheduler_weight
SCHEDULER_FAIRNESS_WINDOW	int	300	Fairness tracking window (seconds)
SCHEDULER_DEPRIORITIZE_THRESHOLD	float	0.5	Usage threshold for deprioritization
SCHEDULER_SCORE_MODEL_LOADED	int	100	Score bonus for pre-loaded model
SCHEDULER_SCORE_LOW_UTILIZATION	int	50	Score bonus for low GPU utilization
SCHEDULER_SCORE_LATENCY	int	40	Score factor for low latency
SCHEDULER_SCORE_SHORT_QUEUE	int	30	Score factor for short queue
SCHEDULER_SCORE_HIGH_THROUGHPUT	int	20	Score factor for high throughput

Latency Tracking

Variable	Type	Default	Description
LATENCY_EMA_ALPHA	float	0.3	EMA smoothing factor
LATENCY_EMA_PERSIST_INTERVAL	int	30	EMA persistence interval (seconds)

Backend Registry

Variable	Type	Default	Description
BACKEND_POLL_INTERVAL	int	30	Health check interval (seconds)
BACKEND_HEALTH_TIMEOUT	int	5	Health check timeout (seconds)
BACKEND_UNHEALTHY_THRESHOLD	int	3	Failed checks before marking unhealthy
BACKEND_CIRCUIT_BREAKER_THRESHOLD	int	3	Failures before circuit opens
BACKEND_CIRCUIT_BREAKER_RECOVERY_SECONDS	int	30	Circuit breaker recovery time
BACKEND_ADAPTIVE_POLL_FAST_INTERVAL	int	10	Fast poll interval after unhealthy
BACKEND_ADAPTIVE_POLL_FAST_DURATION	int	120	Duration of fast polling (seconds)

Request Handling

Variable	Type	Default	Description
MAX_REQUEST_SIZE	int	52428800	Max HTTP request body (50 MB)
BACKEND_REQUEST_TIMEOUT	int	300	Total request timeout (seconds)
BACKEND_REQUEST_TIMEOUT_PER_ATTEMPT	int	180	Per-attempt timeout (seconds, high for large model prefills)
BACKEND_RETRY_MAX_ATTEMPTS	int	3	Max total retry attempts
STRUCTURED_OUTPUT_RETRY_ON_INVALID	bool	true	Retry on invalid structured output

Logging

Variable	Type	Default	Description
----------	------	---------	-------------

Variable	Type	Default	Description
LOG_LEVEL	str	INFO	Log level (DEBUG, INFO, WARNING, ERROR, CRITICAL)
LOG_FORMAT	str	json	Log format (json or text)
LOG_FILE	str	None	Log file path (optional, stdout if not set)

Audit Logging

Variable	Type	Default	Description
AUDIT_LOG_ENABLED	bool	true	Enable audit logging
AUDIT_LOG_PROMPTS	bool	true	Log user prompts
AUDIT_LOG_RESPONSES	bool	true	Log LLM responses

Telemetry & GPU

Variable	Type	Default	Description
TELEMETRY_RETENTION_DAYS	int	30	Telemetry data retention period
TELEMETRY_CLEANUP_INTERVAL	int	3600	Cleanup interval (seconds)
SIDECAR_TIMEOUT	int	5	Sidecar HTTP call timeout (seconds)

Observability

Variable	Type	Default	Description
METRICS_ENABLED	bool	true	Enable Prometheus metrics
METRICS_PREFIX	str	mindrouter	Metrics name prefix
OTEL_ENABLED	bool	false	Enable OpenTelemetry
OTEL_EXPORTER_OTLP_ENDPOINT	str	None	OpenTelemetry exporter endpoint

CORS

Variable	Type	Default	Description
CORS_ORIGINS	list	["http://localhost:3000", "http://localhost:8000"]	Allowed origins (JSON array or comma-separated)

Chat UI

Variable	Type	Default	Description
CHAT_FILES_PATH	str	/data/chat_files	Chat file upload directory
CHAT_UPLOAD_MAX_SIZE_MB	int	10	Max upload file size (MB)
CHAT_UPLOAD_ALLOWED_EXTENSIONS	list	See below	Allowed upload file extensions

Default allowed extensions: .txt, .md, .csv, .json, .html, .htm, .log, .docx, .xlsx, .pdf, .jpg, .jpeg, .png, .gif, .webp

Tokenizer

Variable	Type	Default	Description
DEFAULT_TOKENIZER	str	cl100k_base	Default tokenizer encoding

Status Enums Reference

MindRouter uses the following status enumerations throughout the system:

BackendStatus

- **HEALTHY** — Backend is online and accepting requests
- **UNHEALTHY** — Backend is failing health checks
- **DISABLED** — Backend has been manually disabled by an admin
- **DRAINING** — Backend is finishing in-flight requests but not accepting new ones
- **UNKNOWN** — Backend status has not yet been determined

NodeStatus

- **ONLINE** — Node sidecar is reachable and reporting metrics
- **OFFLINE** — Node sidecar is unreachable
- **UNKNOWN** — Node status has not yet been determined

RequestStatus

- **QUEUED** — Request is waiting for a backend slot
- **PROCESSING** — Request is being handled by a backend
- **COMPLETED** — Request finished successfully
- **FAILED** — Request encountered an error
- **CANCELLED** — Request was cancelled (e.g., client disconnect)

Production Security Hardening

When deploying MindRouter to production, apply the following security measures:

- **Secure session cookies** — Set `SESSION_COOKIE_SECURE=True` to ensure session cookies are only sent over HTTPS.
- **Security headers** — Add `Strict-Transport-Security` (HSTS), `X-Frame-Options`, and `Content-Security-Policy` (CSP) headers at the reverse proxy layer (e.g., nginx).
- **Restrict CORS origins** — Set `CORS_ORIGINS` to only the specific domains that need API access. Do not use `*` in production.

Transparent Data Encryption (TDE)

MindRouter enables MariaDB Transparent Data Encryption to protect data at rest. TDE encrypts all InnoDB tablespaces, redo logs, and temporary files on disk using AES-256. Data is decrypted transparently when accessed through the MySQL protocol — no application code changes required.

What TDE protects

- InnoDB data files (`.ibd`) containing chat messages, audit logs, and attachment text
- InnoDB redo logs (write-ahead logs)

- Temporary files created during query execution

What TDE does NOT protect: Anyone with valid database credentials can still read plaintext data via SQL queries. TDE guards against stolen disks, leaked backups, and unauthorized filesystem access.

Generating encryption keys

```
# From the project root:
bash scripts/generate_encryption_keys.sh
```

This generates independent key sets for the primary and archive databases under `mariadb/encryption/`. The script is idempotent — it skips generation if keys already exist.

Key backup

Critical: Back up the encryption key files to a secure offline location separate from the database backups. Without the keys, encrypted data is **permanently unrecoverable**.

- `mariadb/encryption/primary/keyfile.key + keyfile.enc`
- `mariadb/encryption/archive/keyfile.key + keyfile.enc`

Verifying encryption is active

```
# Check which tablespaces are encrypted (ENCRYPTION_SCHEME=1 means encrypted):
SELECT NAME, ENCRYPTION_SCHEME
FROM information_schema.INNODB_TABLESPACES_ENCRYPTION
WHERE ENCRYPTION_SCHEME = 1;
```

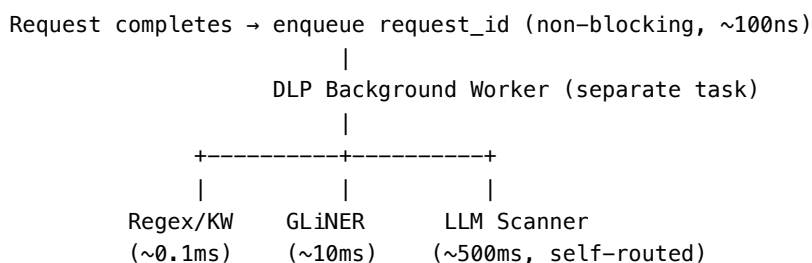
After enabling TDE and restarting MariaDB, background encryption threads will encrypt existing tablespaces in-place. This happens automatically and does not block normal queries.

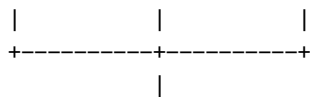
Data Loss Prevention (DLP)

MindRouter includes a built-in DLP system that scans prompts and responses for sensitive data including PII, HIPAA/PHI, FERPA, PCI, and CUI. Scanning happens **asynchronously after the response is delivered**, so there is zero impact on inference latency. DLP is a passive, bump-on-the-wire system — it never blocks, delays, or modifies requests or responses.

Architecture

After a request completes and the response is sent to the client, the request ID is enqueued (non-blocking) to a bounded in-memory queue. A background worker drains the queue and runs the configured scanners against the stored request/response text. If sensitive data is found, an alert is written to the `dlp_alerts` table and email notifications are sent if configured.





Merge findings → Classify severity → Write alert → Email

Scanning Tiers

Tier	Engine	Latency	Detects	Best for
Regex & Keywords	Built-in regex patterns + custom patterns + keyword list	~0.1ms	SSNs, credit cards, emails, phone numbers, custom patterns	Known formats with fixed structure. High precision, no false positives.
GLiNER NER	Neural entity recognition (<code>urchade/gliner_multi_pii-v1</code> , CPU)	~10ms	Names, dates of birth, passport/license numbers, medical records, student IDs	Entities in natural language that don't match fixed patterns. Zero-shot — can detect custom entity types by name.
LLM Contextual	Dedicated LLM, self-routed via <code>/v1/chat/completions</code>	~500ms	FERPA violations, HIPAA/PHI context, CUI, semantic sensitive data	Contextual understanding — e.g. detecting that "John got a B+ in CS101" is a FERPA violation even though no single word is PII.

Severity Levels

Level	Meaning	Default categories	Action
Major	High-risk data exposure	SSN, credit card, bank account, passport, medical record	Immediate attention. Email sent to major recipients.
Moderate	Potential compliance issue	Driver license, date of birth, student ID	Review recommended. Email sent to moderate recipients.
Minor	Low-risk, informational	Person name, email address, phone number	Logged for audit. Email sent to minor recipients (if configured).

The highest severity among all findings in a single scan determines the alert level. Categories not mapped in severity rules default to **Moderate**.

Admin Configuration

Configure DLP in the admin dashboard at **Admin → DLP**. Each section has a  help button with detailed instructions and examples.

- **Master toggle** — Enable or disable all DLP scanning. When disabled, the background worker skips all processing (zero overhead).
- **Regex & Keyword Scanner** — Toggle the fast pattern matcher. Add custom regex patterns with a visual builder (name, regex, category, severity). Add keywords one per line.
- **GLiNER NER Scanner** — Toggle neural entity recognition. Adjust the confidence threshold (0.1–0.95). Check/uncheck built-in entity categories. Add custom entity categories for domain-specific detection (e.g. "vandal id", "grant number").
- **LLM Contextual Scanner** — Toggle LLM-based semantic analysis. Choose the model (e.g. `qwen/qwen3.5-4b`). Customize the system prompt for your compliance regime (FERPA, HIPAA, CUI).
- **Severity Rules** — Visual mapping of entity categories to severity levels. Add new mappings with the inline form. Remove mappings you don't need.

- **Email Notifications** — Set email recipients per severity level (comma-separated). Requires SMTP to be configured in Settings.

Custom Entity Categories

You can extend DLP detection beyond the built-in categories:

- **Regex patterns** — Click "Add Pattern" in the Regex scanner section. Provide a name, Python regex, category name, and default severity. Example: Name="Vandal ID", Regex=`\bV\d{8}\b`, Category="vandal id".
- **GLiNER categories** — Type a descriptive category name in the "Add custom category" field and click Add. GLiNER is zero-shot and can detect entity types by description alone.
- **Severity rules** — After adding a custom category, add a corresponding severity rule so the system knows how to classify findings in that category.

Self-Routing Loop Prevention

When the LLM scanner is enabled, MindRouter automatically creates a dedicated internal API key. The LLM scanner uses this key to call `/v1/chat/completions` on itself. The DLP worker checks the API key on each incoming request — if it matches the internal key, the scan is skipped. This prevents an infinite scanning loop where the DLP scan triggers another DLP scan.

Alerts & Acknowledgment

All DLP findings are stored in the `dlp_alerts` table and displayed on the Admin → DLP page. Alerts can be filtered by severity, scanner type, and free-text search. Admins can acknowledge alerts to mark them as reviewed. Stats cards show 24-hour alert counts, severity breakdown, average scan latency, and unacknowledged count. Auditors have read-only access to the alerts table.

Performance & Reliability

- **Zero latency impact** — The only code in the request path is `queue.put_nowait(request_id)` (~100ns) wrapped in try/except. It cannot slow down or break inference.
- **Bounded queue** — The queue holds up to 10,000 items. If full (e.g. during a traffic spike), new items are silently dropped with a log warning. Best-effort scanning.
- **Crash isolation** — The DLP worker runs as an independent asyncio task with its own DB session. Any crash is caught, logged, and the worker continues processing the next item.
- **GLiNER lazy loading** — The ~200MB model loads on first scan (~2-3s), then stays in memory. Subsequent scans take ~10ms on CPU.
- **Text truncation** — The LLM scanner truncates input to 8,000 characters to limit token usage and cost.
- **Image content skipped** — Multimodal image inputs are automatically excluded from scanning. Only text content (messages, prompts, responses) is scanned.

Deployment

MindRouter is designed for deployment on Linux servers with NVIDIA GPUs. The full deployment guide covers:

- Rocky Linux 8 prerequisites and dependency installation
- SSL/TLS configuration (self-signed and Let's Encrypt)
- Apache reverse proxy setup
- Firewall and SELinux configuration
- Docker Compose production stack

- Database migrations
- GPU sidecar agent deployment
- Node and backend registration
- Verification and ongoing operations

Docker Compose Services

The production stack consists of the following containers:

Service	Port	Description
app	8000	Main FastAPI application (multi-worker)
mcp	8001	MCP SSE server (single-worker — required for session affinity)
mariadb	3306	Primary MariaDB database
mariadb-archive	3307	Archive database for tiered data retention
redis	6379	Redis for token counters and caching

The `mcp` service runs the [MCP SSE server](#) as a dedicated single-worker process. The MCP SSE transport stores sessions in memory, so it must run with exactly one worker to ensure that SSE connections and subsequent POST messages are handled by the same process. The main `app` service proxies all `/mcp/*` requests to this container transparently.

GPU Sidecar Deployment

The GPU sidecar agent runs on each inference node to expose per-GPU hardware metrics and enable auto-discovery of inference endpoints. Build and deploy directly from GitHub — no need to clone the repository.

Create the env file (once per node)

```
sudo mkdir -p /etc/mindrouter
python3 -c "import secrets; print('SIDEAR_SECRET_KEY=' + secrets.token_hex(32))" | sudo tee /etc/mind
sudo chmod 600 /etc/mindrouter/sidecar.env
```

Build a specific version

```
docker build -t mindrouter-sidecar:v2.0.0 \
  -f Dockerfile.sidecar \
  https://github.com/ui-insight/MindRouter.git#v2.0.0:sidecar
```

Build latest from master

```
docker build -t mindrouter-sidecar:latest \
  -f Dockerfile.sidecar \
  https://github.com/ui-insight/MindRouter.git:sidecar
```

Run the sidecar

```
# Run bound to localhost only (nginx will proxy external traffic)
docker run -d --name gpu-sidecar \
  --gpus all \
  -p 127.0.0.1:18007:8007 \
  --env-file /etc/mindrouter/sidecar.env \
  --restart unless-stopped \
  mindrouter-sidecar:v2.0.0
```

Upgrade to a new version

```
docker build -t mindrouter-sidecar:v2.0.0 \
  -f Dockerfile.sidecar \
  https://github.com/ui-insight/MindRouter.git#v2.0.0:sidecar
docker stop gpu-sidecar && docker rm gpu-sidecar
docker run -d --name gpu-sidecar \
  --gpus all \
  -p 127.0.0.1:18007:8007 \
  --env-file /etc/mindrouter/sidecar.env \
  --restart unless-stopped \
  mindrouter-sidecar:v2.0.0
```

In production, bind the sidecar to localhost only (as shown above) and use an nginx reverse proxy on port 8007 to handle external traffic. The env file at `/etc/mindrouter/sidecar.env` persists the secret key across upgrades — generate it once per node and it's reused automatically.

Nginx Reverse Proxy Configuration

Each inference node runs an nginx reverse proxy that terminates TLS and forwards requests to the locally-bound inference engine (Ollama or vLLM) and GPU sidecar. This is required because inference engines bind to `127.0.0.1` for security — nginx handles external TLS traffic on the public-facing ports.

Upload size for multimodal requests

Multimodal inference requests (images sent to vision models like Qwen3-VL, gemma3, LLaVA, etc.) include base64-encoded image data in the JSON body, which can be significantly larger than text-only requests. A typical high-resolution image produces 1–10MB of base64 data. Nginx's default `client_max_body_size` is only 1MB, which will cause **413 Request Entity Too Large** errors for any image upload.

You must set `client_max_body_size` in every nginx proxy config that handles inference traffic (Ollama and vLLM proxies). A value of `50m` (50MB) is recommended to accommodate large images and multi-image requests:

Example: Ollama proxy config

```
# /etc/nginx/conf.d/ollama-proxy.conf
server {
    listen 8001 ssl;
    listen [::]:8001 ssl;
    client_max_body_size 50m;    # Required for multimodal image uploads
    server_name node.example.edu;

    ssl_certificate      /etc/pki/tls/certs/node.example.edu.crt;
    ssl_certificate_key  /etc/pki/tls/private/node.example.edu.key;

    location / {
        proxy_pass http://127.0.0.1:18001;
        proxy_set_header Host $host;
```



```

        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 300s;    # Long timeout for inference
        proxy_send_timeout 300s;
    }
}

```

Example: vLLM proxy config

```

# /etc/nginx/conf.d/vllm-proxy.conf
# Multiple server blocks, one per vLLM instance on different ports
server {
    listen 8001 ssl;
    client_max_body_size 50m;    # Required for multimodal image uploads
    server_name node.example.edu;

    ssl_certificate      /etc/pki/tls/certs/node.example.edu.crt;
    ssl_certificate_key  /etc/pki/tls/private/node.example.edu.key;

    location / {
        proxy_pass http://127.0.0.1:18001;
        proxy_set_header Host $host;
        proxy_read_timeout 300s;
        proxy_send_timeout 300s;
    }
}

```

Important: This setting must be applied on every inference node in the cluster, not just nodes currently running vision models. Any node may have a multimodal model deployed in the future, and missing this setting will cause silent failures (the request reaches MindRouter, gets routed to the backend, but nginx rejects it before it reaches the inference engine).

Verifying the setting

```

# Check all nginx proxy configs across the cluster:
for host in marten lynx webbyg1 webbyg2 calvin eunice aspen1 aspen2 aspen3 aspen4 aspen5; do
    echo -n "$host: "
    ssh $host "grep -r client_max_body_size /etc/nginx/conf.d/ 2>/dev/null || echo 'NOT SET - needs fi
done

```

Testing

MindRouter has a comprehensive test suite covering unit, integration, end-to-end, smoke, stress, and accessibility tests.

Quick Reference

Command	Description
<code>make test-unit</code>	Run unit tests (525+ tests)
<code>make test-int</code>	Integration tests (requires live backends)
<code>make test-e2e</code>	End-to-end tests
<code>make test-smoke</code>	Smoke tests (full API surface)

Command	Description
<code>make test-stress</code>	Load/stress tests
<code>make test-a11y</code>	WCAG 2.1 accessibility tests
<code>make test-sidecar</code>	GPU sidecar agent tests
<code>make test-all</code>	Run all test suites

MindRouter - LLM Inference Load Balancer

Powered by [RCDS](#)

 This project was developed with support from the National Science Foundation ([NSF Award #2427549](#))